

Web Technologies

Web services

Web application
development via REST



Prof. Sabin Corneliu Buraga – profs.info.uaic.ro/sabin.buraga/
Assoc. Prof. Andrei Panu – profs.info.uaic.ro/andrei.panu/

„Never say «it's not possible»,
but start with «let's see».”

Nicolae Iorga



how can the Web resources be accessed and processed



what are Web services



how can we implement a service



an architectural model for developing services



a pragmatic way for developing and invoking Web services



a methodology regarding the development of Web services (APIs) aligned with the REST paradigm



practical aspects of interest for developers



accessing resource representations through REST



What are the Web's purposes?

Creating and interacting
with an inter-human communication space

► knowledge sharing

social Web (“Web 2.0”) – users as prosumers
(information producers and consumers)

Creating and interacting
with an inter-human communication space

► knowledge sharing

Web of data / semantic Web

modeling and interconnecting (linked)
data/information/knowledge to be “understood” and
processed “intelligently” by computational entities

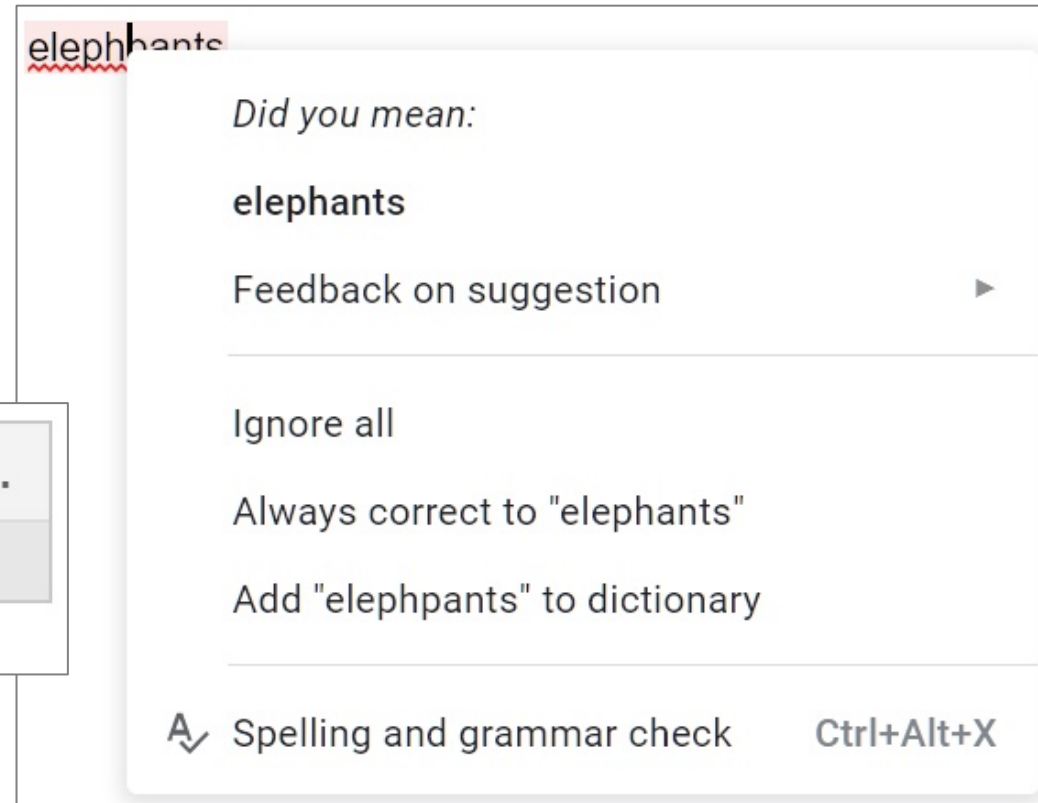
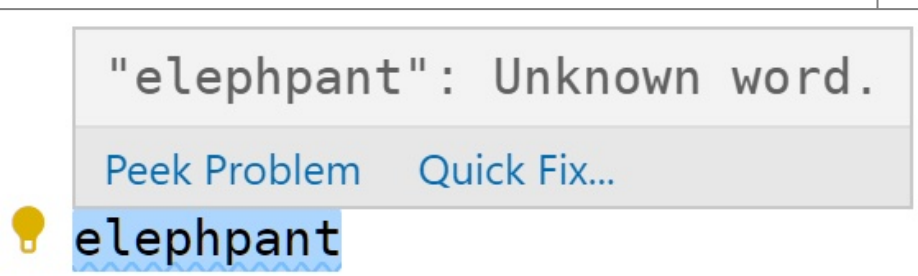
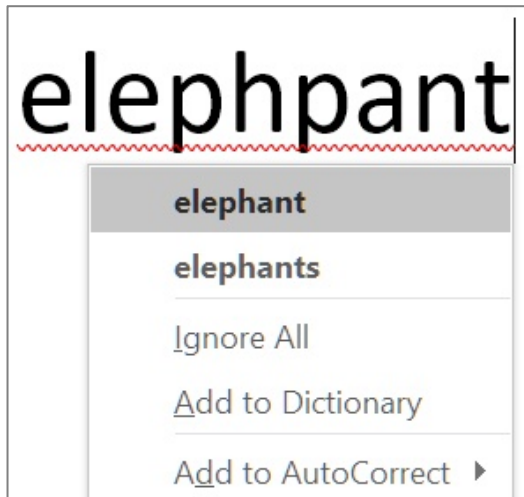
Make use of computational power

the access to the Web can be realised
by using devices with limited resources

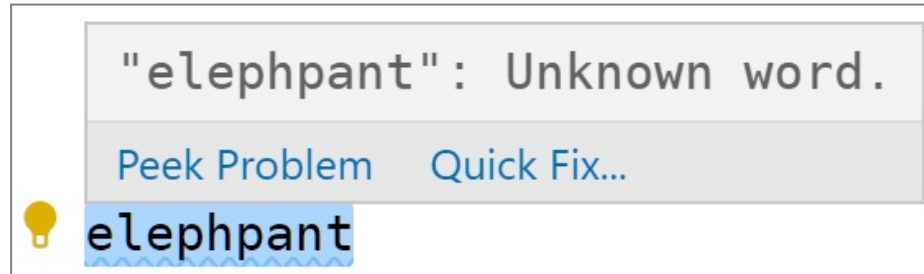
ubiquitous Web (omnipresent):
mobile Web, 3D Web, WoT – Web of Things,...



How can the resources
– data, information, knowledge –
available on the Web be
accessed and processed?



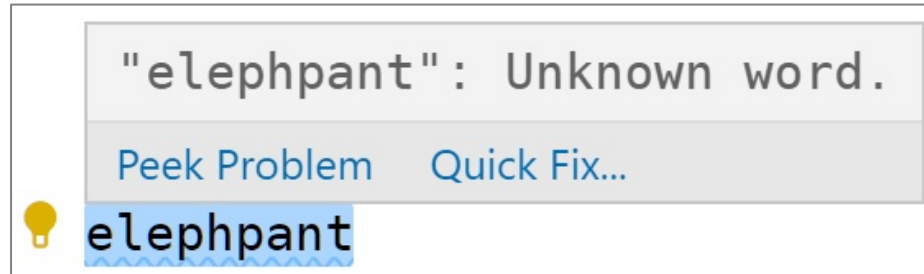
real solution: the multitude of different programs
– usually, included in other applications
(e.g., Firefox, Google Docs, Visual Studio Code, Word) –
used to check spelling (spelling checkers)



the functionalities/data cannot be shared with other native
(local) and/or Web (remote) applications

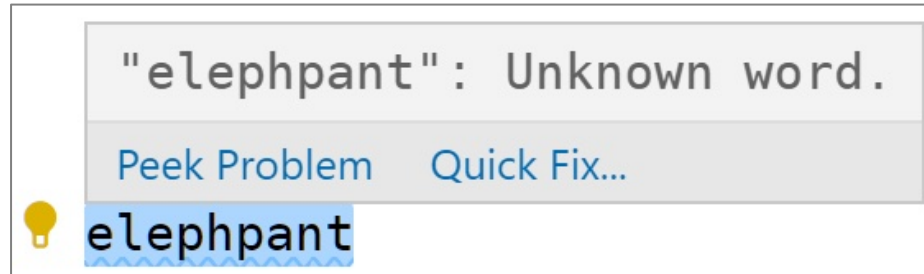
we cannot (re)use a program in another context
– not even on the same platform

(for example, spelling checking in Firefox based on the tool
provided by Visual Studio Code)



each tool has its own dictionary modeled in a specific format, proprietary or not

thus, the data/knowledge cannot be accessed, modified, and/or reused by third party applications



a spelling checker is **tightly coupled**
by the parent application

it cannot be used independently

Web developers' needs

Multi-platform, loosely-coupled solutions

Internet/Web-based integration (in real-time)
of applications, services, and systems

Web developers' needs

Multi-platform, loosely-coupled solutions

example:

finding service offerings,
based on user's geographic location,
in the context of mobile devices

scenario:

“I want to order     available nearby, products that will be delivered to my home as soon as possible”

Web developers' needs

Multi-platform, loosely-coupled solutions

data should be described in order to be “understood”
by computers and to be easily interconnected

Web “puzzles”

“intelligently” interconnecting multiple services providing data of
interest, according to the user preferences

example: predicting the appearance of a new version of a game and
its price depending on the current context

Web developers' needs

Pluggable & versatile services

software (Web applications) considered services
Software as a Service – SaaS

provider of services used at application level
Application Service Provider – ASP

solution

Dividing applications into – independent – services
that can be composed, meant to connect and
orchestrate in a spontaneous way
in the business/technical processes

Web component-based software

solution

“The Web is the computer”

the availability of (an) architecture(s) that...

provide(s) support for communication paradigms
– based on current Web technologies –
between heterogeneous applications

which use different paradigms & programming languages,
and/or hardware/software platforms

solution

“The Web is the computer”

the availability of (an) architecture(s) that...

allow(s) transparent localisation of services

solution

“The Web is the computer”

the availability of (an) architecture(s) that...

facilitate(s) adding, replacing, removing services
in a dynamic way

solution

“The Web is the computer”

the availability of (an) architecture(s) that...

hide(s) the system details from the developer

solution

“The Web is the computer”

the availability of (an) architecture(s) that...

ensure(s) the quality of distributed and/or parallel
application development & deployment:
**standardization, security, availability,
reusability, maintainability, etc.**



What are Web services?

web services

“A **service** is an abstract resource that represents a capability of performing tasks that form a **coherent functionality** from the point of view of providers entities and requesters entities.”

www.w3.org/TR/ws-gloss/

web services

“A **Web service** is a software system designed to support interoperable machine-to-machine interaction over a network.”

www.w3.org/TR/ws-gloss/

web services

Software providing a specific functionality

resource access – Instagram, GoodReads, Pinterest, Vimeo

news aggregation – Feedly, Raindrop.io, Reddit

mapping – Google Maps, OpenStreetMap, etc.

instant messaging – Discord, Telegram, Twilio

processing – Amazon Rekognition, IBM Watson,
Microsoft Azure Cognitive Services, Google TensorFlow...

Web statistics – e.g., Open Web Analytics

spelling checking – Bing Spell Check, xSpell,...

data storage – Amazon S3, Dropbox, OneDrive, etc.

content creation via *Generative Artificial Intelligence*

...



All sites ▾ Any time ▾

? Search tips

M**Medium Programming**[latest](#) • [day](#) • [week](#) • [month](#)**L****Lobsters**[latest](#) • [day](#) • [week](#) • [month](#)**Hacker Noon**[latest](#) • [day](#) • [week](#) • [month](#)

- 18h [The Upgrade Hell](#)
- 2d [The Unauthorized Experiment: How AI Secretly Infiltrated Reddit and Changed Users' Minds](#)
- 2d [Colibri: The Fully Declarative And Turing-Complete Language Lurking Inside Swift's Type System](#)
- 2d [CSS Isn't Broken. You Just Never Read the Docs.](#)
- 3d [De Profundis: Month 1 of 12 in My Year of Deliberate Pain with Rust](#)
- 3d [Stop Hiding Bad Code Behind Good Tooling](#)
- 4d [From Color Blindness Grit to a Simple Tool for Change](#)

- 2h [Odin, A Pragmatic C Alternative with a Go Flavour](#)
- 2h [Augmentation / Replacement](#)
- 4h [Stop Fighting Your ROS 2 Environment: Build Faster, Reproducibly, Anywhere](#)
- 4h [What are you doing this week?](#)
- 7h [Fuzzing with Grammars](#)
- 8h [modern-latex: A short guide to LaTeX that avoids legacy craft](#)
- 12h [iBeacons](#)
- 15h [Tech Companies Apparently Do Not Understand Why We Dislike AI](#)

- 8h [The TechBeat: Celebrating Vitruvian Man Day at SDG: A Tribute to Arts, Science, & Human Potential \(5/5/2025\)](#)
- 10h [How to Set Up a Free Oracle Cloud VM for Web Development \(2025 Guide\)](#)
- 10h [Spam vs Phishing: Know the Difference](#)
- 10h [Ukraine is a Building a Drone Wall Against Russia](#)
- 10h [Six Side Characters That Stole The Show In TV Shows](#)
- 11h [A Beginner's Guide to Reading and Writing Smart Contracts on Explorers](#)

used – remotely – by other applications/services

Web Application Development x			
fx	=GOOGLETRANSLATE(B1, "ro", "en")		
	A	B	
1	Text	B2	voltarea aplicațiilor Web
2	Translation	=GOOGLETRANSLATE(B1, "ro", "en")	
3			

web services

Accessed in a standardized way via Web

addressing resources with URI

data transfer via HTTP

messages adopting various data formats: CSV, JSON, XML,...

web services: example

A service provided by a travel agency

offering – and, possibly, selling –
various holiday options



web services: example

A service provided by a travel agency

uses other (software) services
available on the Web

mapping + weather services

accommodation services

financial transactions– e.g., e-banking

transport services

social recommendation services



How can we implement a service?

web services

Standard implementation

using Web application servers + frameworks



ASP.NET MVC, Django, Node.js,
PHP (Laravel, Symfony, Yii,...),
Play, Ruby on Rails, Spring, etc.

web services

Traditionally, the application provides
an user-interface available on the Web

markup language – e.g., HTML

styles for content presentation – CSS

interactivity via JavaScript (+libraries/frameworks)



Packagist is the main [Composer](#) repository. It aggregates public PHP packages installable with Composer.

afarkas/html5shiv

Defacto way to enable use of **HTML5** sectioning elements in legacy Internet Explorer.

JavaScript 551 355

★ 9 876

tinymce/tinymce

Web based JavaScript **HTML** WYSIWYG editor control.

JavaScript 6 301 170

★ 193

froala/wysiwyg-editor

A beautiful jQuery WYSIWYG **HTML** rich text editor. High performance and modern design make it easy to use for developers and loved by users.

CSS 271 233

★ 5 358

mottie/tablesorter

tablesorter (FORK) is a jQuery plugin for turning a standard **HTML** table with THEAD and TBODY tags into a sortable table without page refreshes. tablesorter can successfully parse and sort man

JavaScript 189 806

★ 2 602

components/handlebars.js

Handlebars.js and Mustache are both logicless templating languages that keep the view and the code separated like we all know they should be.

JavaScript 181 375

★ 50

hhurz/tableexport.jquery.plugin

Export **HTML** table to CSV, DOC, JSON, PDF, PNG, SQL, TSV, TXT, XLS, XLSX, XML

HTML 332

★ 996

Active filters

type: component

Clear all

Package type

aimeos-extension	2
amila-laravel-cms-plugin	1
arikaim-components-library	1
arikaim-library	1
automad-package	1
behat-extension	3
biblioteca	1
bolt-extension	2
bundle	1
cakephp-plugin	34
campaignchain-doc	1
commonmark-extension	6
componenet	1
component	32
composer-package	1

Show more...

Tags

- ☐ html 11
- ☐ javascript 8
- ☐ html5 7
- ☐ wysiwyg 5

the requests are captured via forms,
interactive components + hypermedia links

33	Euro
159.46	Romanian Leu
33	유로
159.48	루마니아 레우



human users should interpret the labels and dialog fields exposed by the Web application's interface

web services

The service implemented by the application
provides a response
(a representation of a Web resource)

usually, an HTML document whose content is transferred
to the client according to a protocol – HTTP(S)

there are rare cases where HTML documents
are written/generated correctly – **malformed markup**
aspect of interest: ignoring syntax errors

web services

How can we obtain the response
in order to be (re)used in our programs?

need: processing data from the HTML code 🪓

Web scraping

usually, specialized libraries/frameworks are used:

Beautiful Soup (Python), **HAP** (C#), **jsoup** (Java),
Masterminds HTML5-PHP, **Parse5** (Node.js),
ScrAPIr (JavaScript), **Scrapy** (Python), **SwiftSoup**

web services

How can we obtain the response
in order to be (re)used in our programs?

```
class="vk_sh c8Zgcf B682Df"><div><span class="DFlfde eNFL1">33</span> <span class="vLqKYe" data-mid="/m/02l6h" data-name="Euro">Euro</span> equals</div><div class="w5HbFd"></div></div><div class="dDoNo ikb4Bb gsrt GDBPqd"><span class="DFlfde SwHCTb" data-precision="2" data-value="164.28225">164,28</span><span class="MWvIVe" data-mid="/m/02zsyq" data-name="Romanian Leu">Romanian Leu</span></div></div><div class="k0Rg6d
```



???

any change in the markups leads to the rewriting of the
program that extracts the data from the HTML document

web services: characterization

Web services make explicit
the implicit specifications

the input data and the result can be (rigorously)
specified by using various validation techniques

examples: JSON Schema, XML Schema

web services: characterization

Used in the interaction between applications

dynamic

lack of a-priori knowledge about the interaction
with other Web applications/services

web services: characterization

Endpoints used for public data processing
– eventually, via open APIs

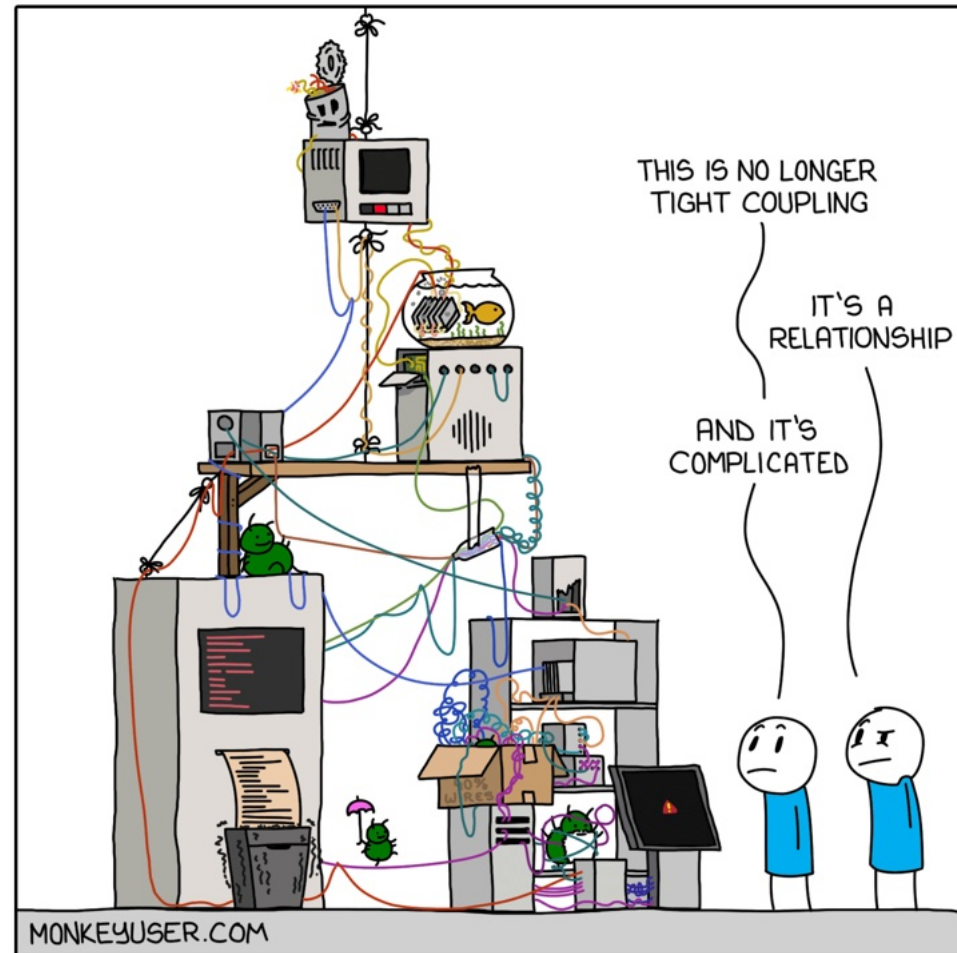
web services: characterization

Developed based on current platforms, architectures, technologies, and languages



Is there an
architectural model
for developing Web
services?

NEXT LEVEL



soa

Service Oriented Architecture

soa

Architectural style for designing and developing
applications considered services
that can be invoked by other applications

T. Erl, *Service-Oriented Architecture.
Analysis & Design for Services and Microservices* (2nd Ed.),
Prentice Hall, 2017

soa

Software development paradigm

adopting the use of services,
that provides functionalities required by users

resources are available through a suite of
independent services whose implementations
do not need to be known (black box)

soa

The components of the system as a whole have a high degree of **independence (de-coupling)**

services must be designed to interact **without dependencies** between them

soa

Services share a **formal contract**

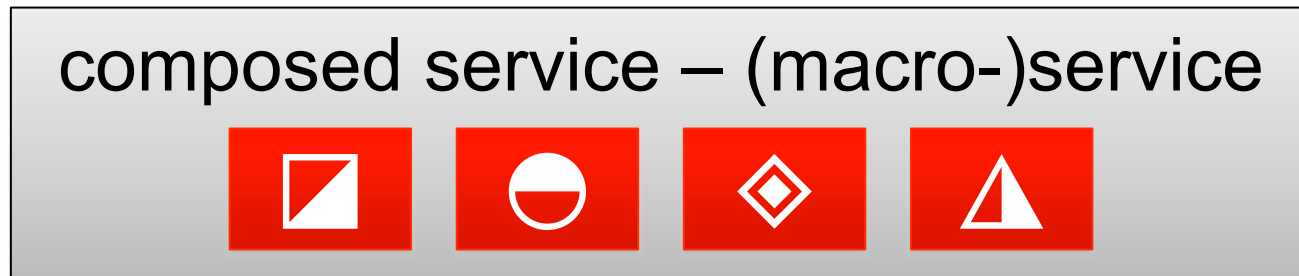
necessity for a formal description of a service:
provided operations (service interface)
data exchange manner (request + response)
service discoverability

soa

The services can be recomposed/orchestrated
according to the requirements
or deployment context

soa

The services can be composed of sub-services
(composable services)
which can be reused (reusable services)



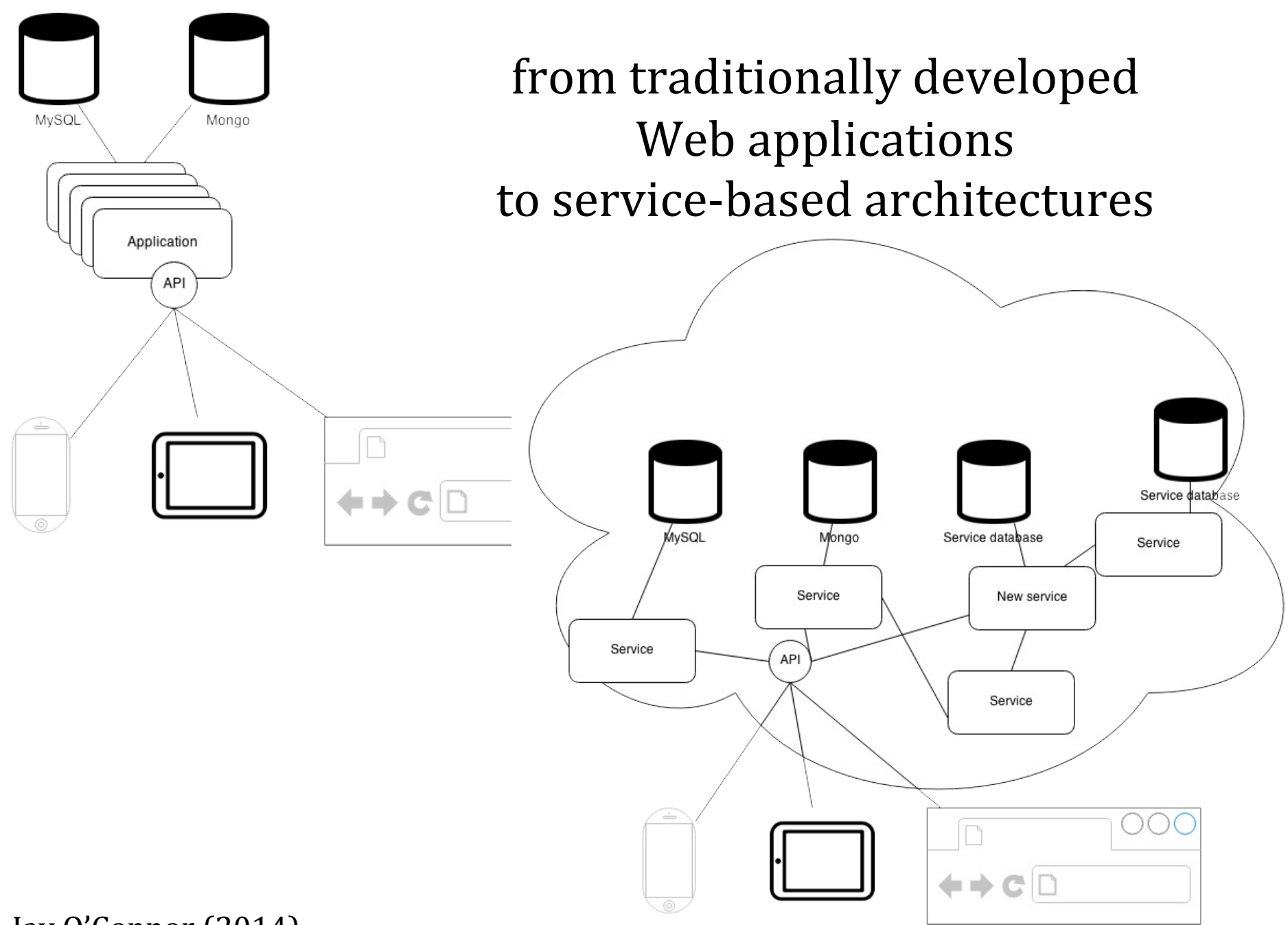
(micro-)services

soa

Services will not depend on the state of communication (statelessness)

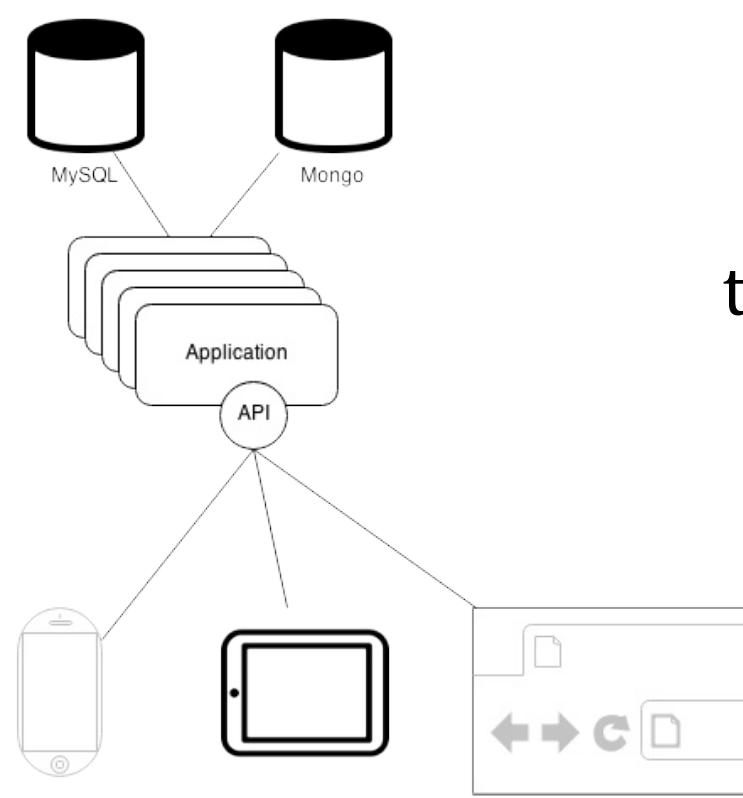
in order to perform an operation, the amount of information that can be kept should be minimal

from traditionally developed Web applications to service-based architectures



Jay O'Connor (2014)

www.bellycard.com/tech-sub/blog/migrating-to-a-service-oriented-architecture-soa/



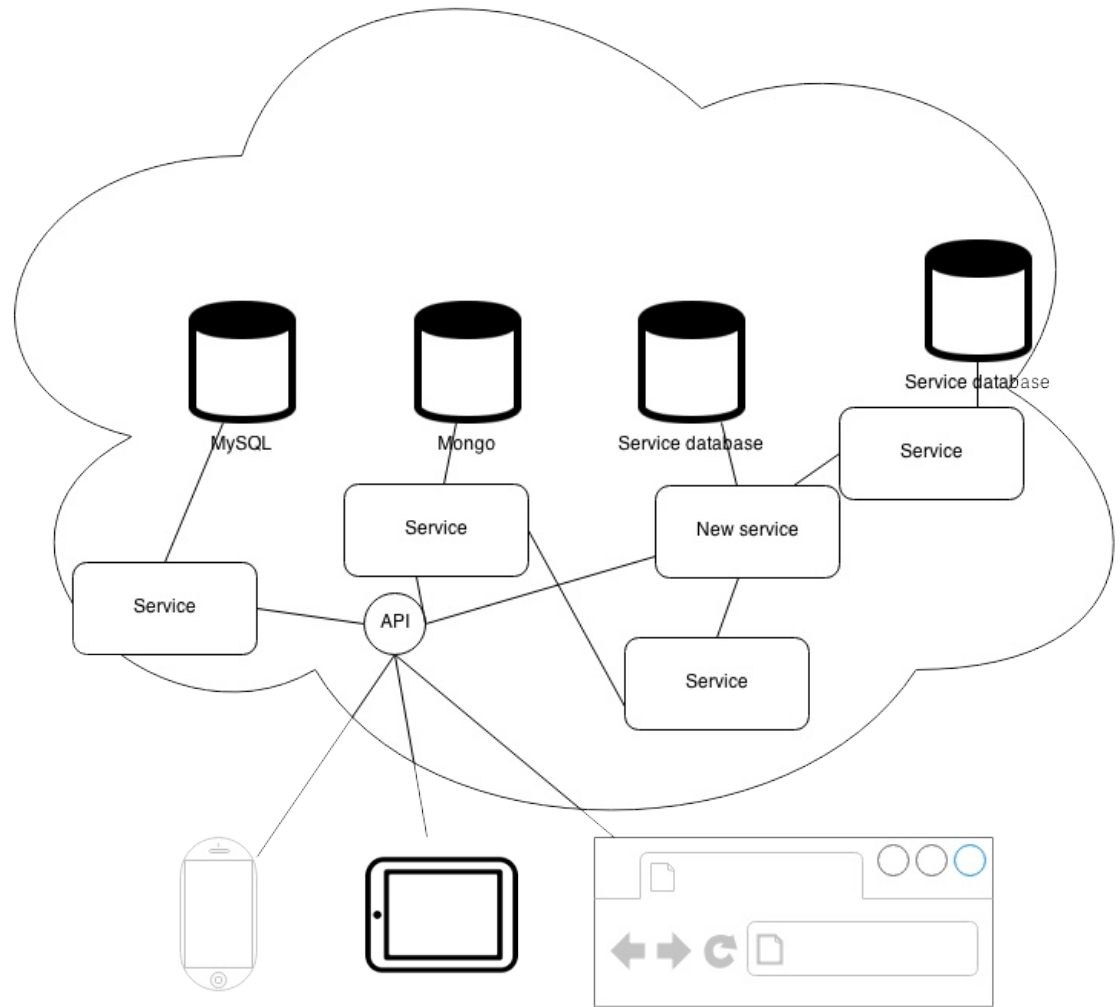
from traditionally developed
Web applications
to service-based architectures

classic approach: the Web application exposes functionalities offered to its potential clients (Web browsers available on multiple platforms) based on data stored persistently – usually, an MV* architectural approach

from traditionally developed
Web applications
to service-based architectures

SOA:

each functionality is
implemented by an
independent service
which can use other
(internal) services and
various data storage
systems



soa

The services - own and/or external -
can be invoked at ...
 server (back-end)
 and/or
 client (front-end)
... level, synchronously or asynchronously

soa

Orchestration

the existence of a main service (**service orchestrator**) that coordinates the invocation of other services and receives the answers provided by them



centralised point of view

official standard (OASIS, 2007):

WS-BPEL (Web Services Business Process Execution Language)

soa

Choreography

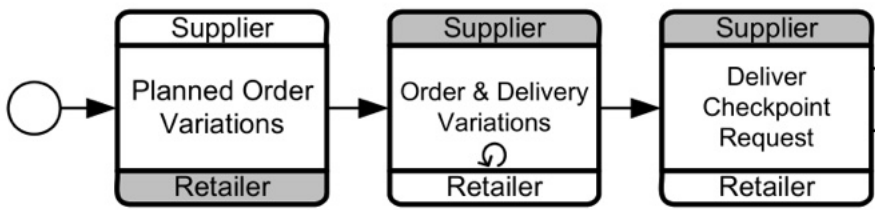
global description of autonomous services participating in an activity, defining the way of exchanging messages, interaction rules (e.g., protocols), and contracts between two or more endpoints



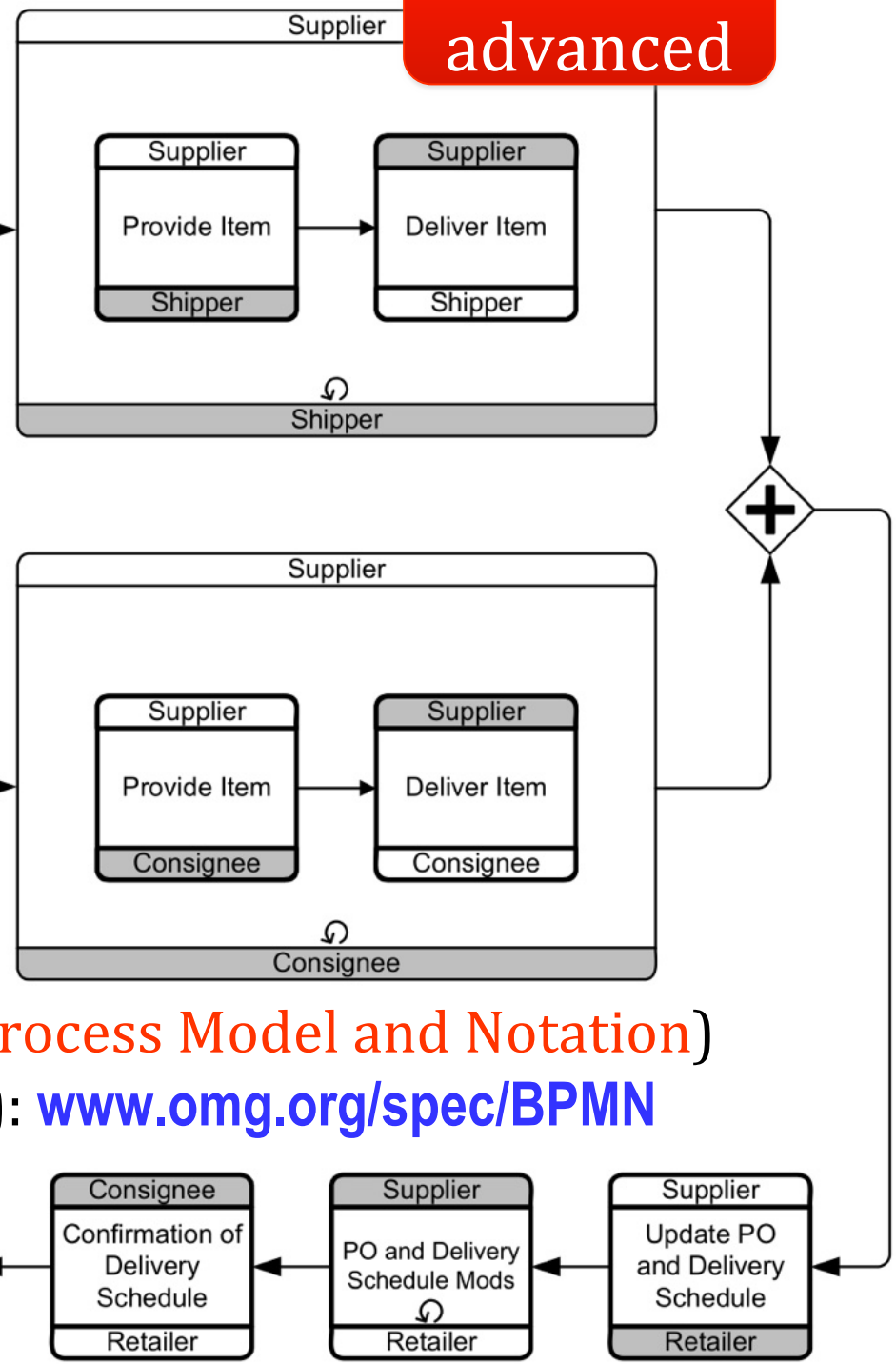
decentralised approach

description: **Web Services Choreography Description Language**

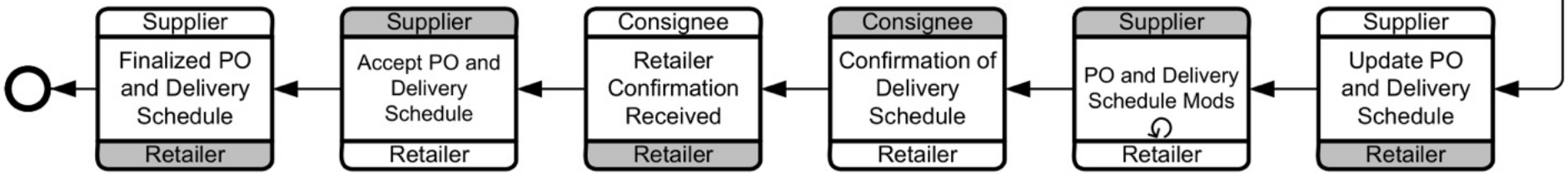
example of a choreography of services considered as processes



here, the provision of goods for a supplier of products



BPMN notation (Business Process Model and Notation)
OMG specification (2014): www.omg.org/spec/BPMN





Web services provider/user



API

micro-/macro-serv.



backend server(s)



virtual machine



legacy systems



workflow engine



core services (calendar, transactions,...)



service context (who, why,...)



service description



discovery services

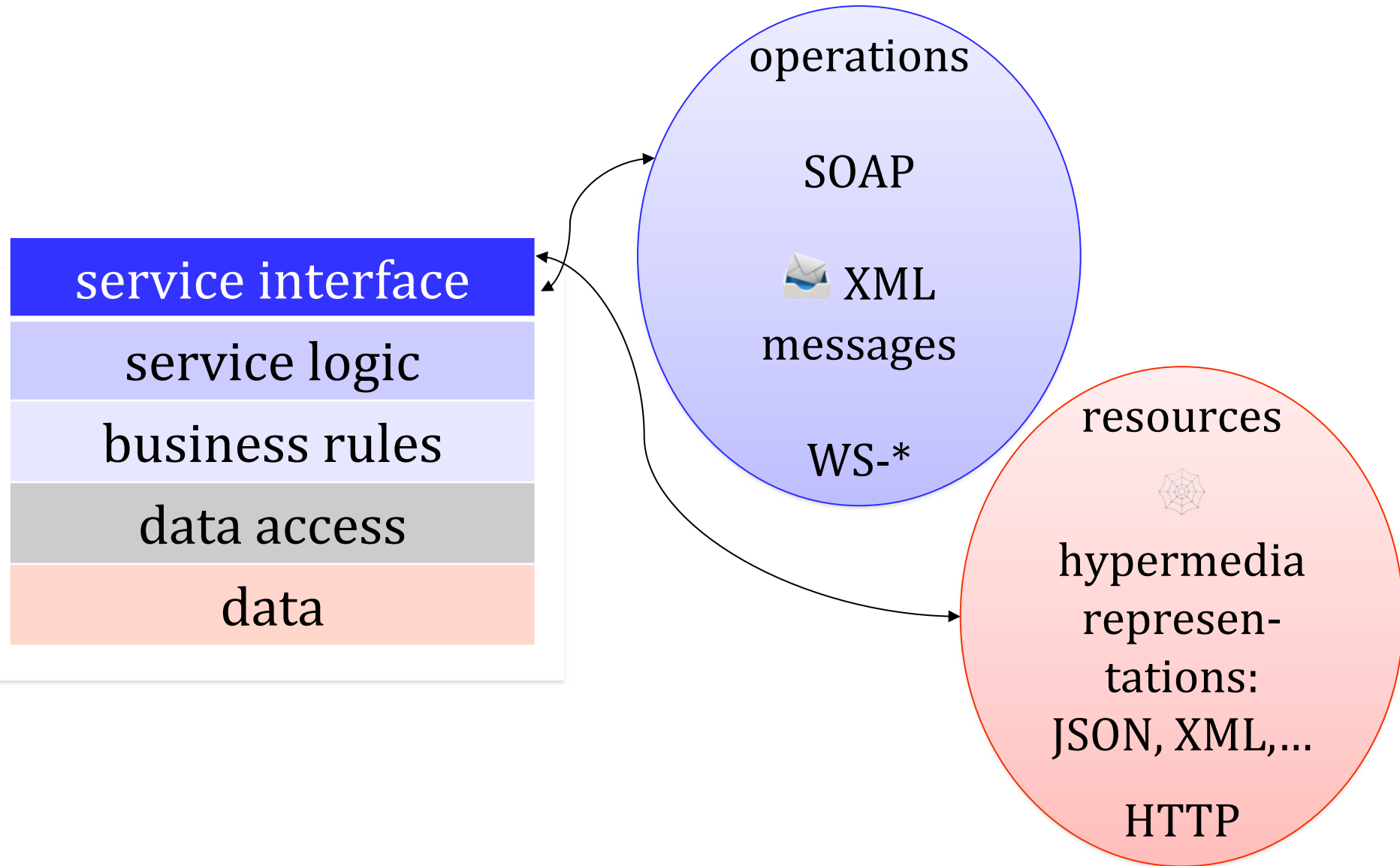


protocols: HTTP(S), WebSocket,...

conceptual layered structure
of a service-based Web platform



Is there a pragmatic way of developing and
invoking Web services?



Web services can be developed via **SOAP** and/or **REST**

S. Tilkov, REST: Not an Intro (2013)

speakerdeck.com/stilkov/rest-not-an-intro-1

rest: representational state transfer

Architectural style for Web application development focused on data representation

Roy Fielding

PhD thesis, 2000 (University of California, Irvine)

www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

rest

The result of a processing leads to obtaining
a **representation** of a resource

rest

The result of a processing leads to obtaining
a **representation** of a resource

Web resource

a user having an account within a system,
a message of a person, a photo, a newsfeed, a software
component, a dataset, a 3D model, ...

rest

The result of a processing leads to obtaining
a **representation** of a resource

representation based on a data format

textual or binary

typical examples – open formats:
HTML, JSON, CSV, PNG, SVG, PDF, etc.

rest

The result of a processing leads to obtaining
a **representation** of a resource

representation based on a data format

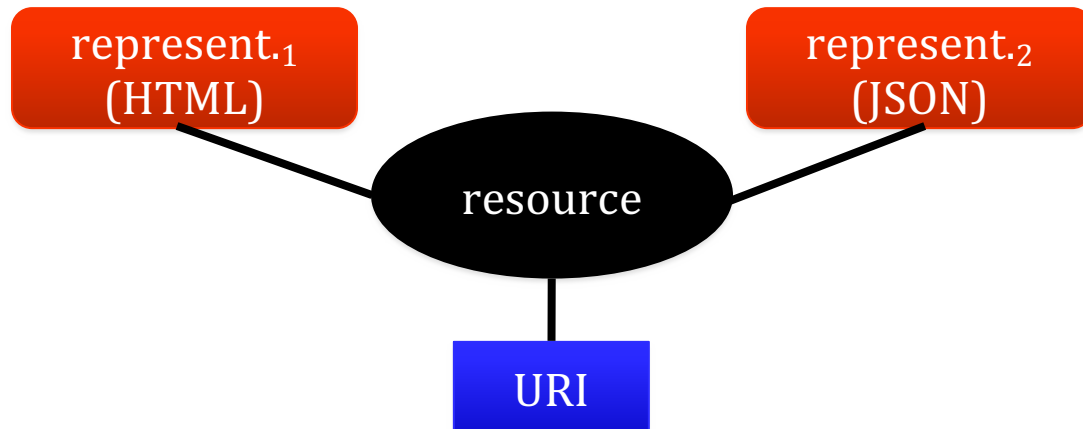
the representation's format is denoted by **MIME types**
text/html, text/xml, text/csv, application/json, image/webp

details in N.Freed et al., Media Types, 16 April 2026
www.iana.org/assignments/media-types/media-types.xhtml

rest

The result of a processing leads to obtaining
a **representation** of a resource

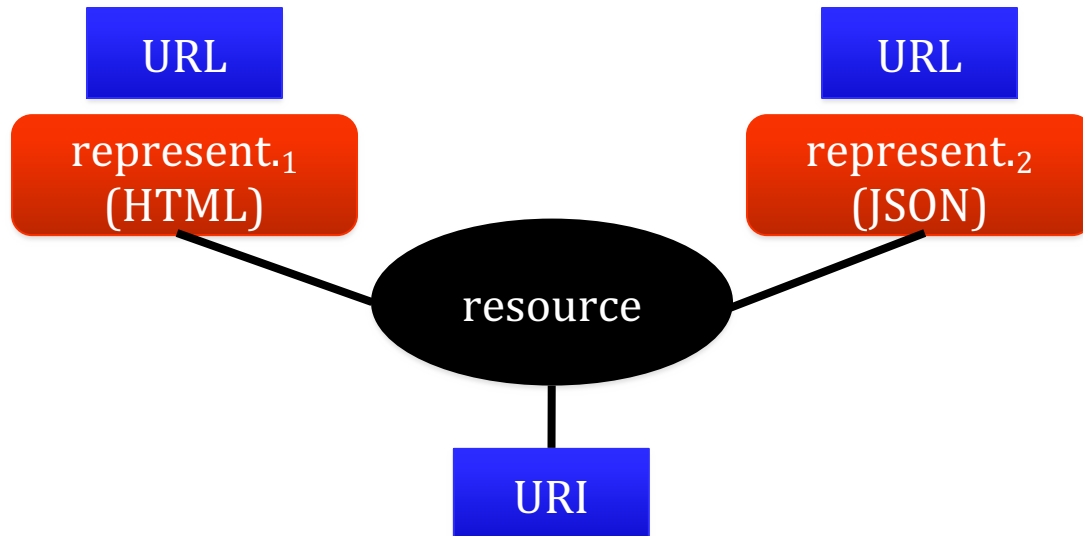
representations of the same resource
– designated by a unique URI – can be multiple



rest

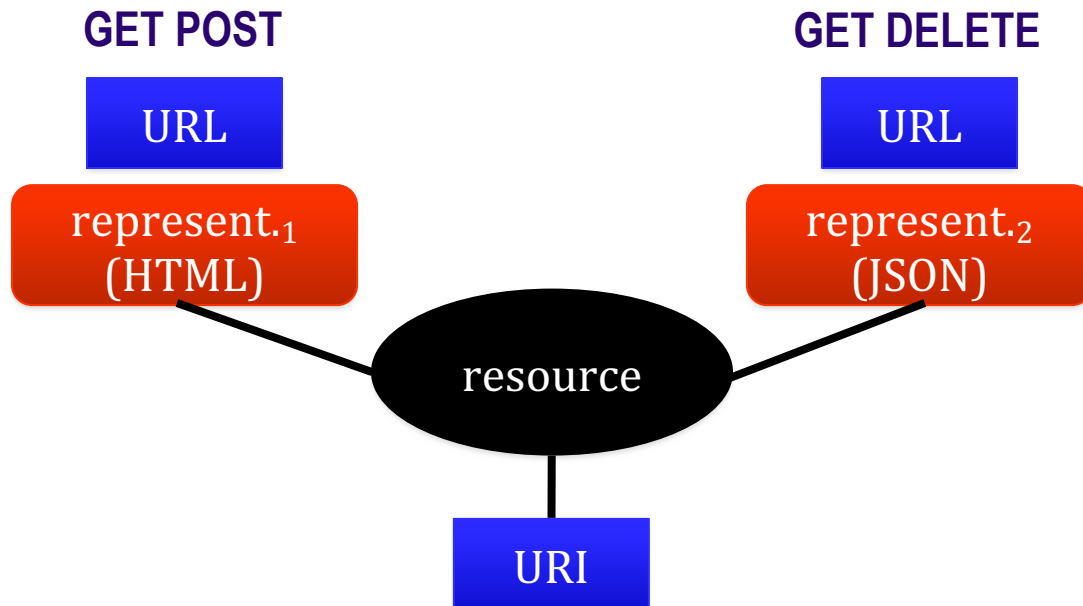
The result of a processing leads to obtaining
a **representation** of a resource

each resource representation has an URL associated



rest

The clients (e.g., Web browsers, robots, players, etc.) interact with resource representations via verbs
“access”: **GET**, “modify”: **POST**, “erase”: **DELETE**,...



rest

The verbs (actions) are stipulated by a protocol



see the 2nd
lecture

! remark:

although usually HTTP is used, in some cases other
protocols can be used also

example: Realtime API OpenAI adopts WebRTC or WebSocket
developers.openai.com/api/docs/guides/realtime

rest

GET

obtains (gets) a resource representation

does not lead to the server's state modification – **safe**

idempotent – identical requests will return the same response (same representation)

*an HTTP method is **idempotent** if the intended effect on the server of making a single request is the same as the effect of making several identical requests –*
developer.mozilla.org/docs/Glossary/Idempotent

rest

HEAD

similar to GET, but provides meta-data only
(does not offer the actual representation)

e.g., last update, content length,...

rest

PUT

replaces (updates) a resource representation
or eventually creates a resource on the Web server
(whose URI is already known)

usually, returns the resource's URI

it is not considered safe, but it is idempotent

rest

PATCH

permits a partial update of a resource representation
(PUT does not provide this feature)

it is not safe, nor idempotent

rest

POST

creates a resource (usually, subordinated to another one)

optionally, additional processing can be performed

it is not safe, nor idempotent

the client does not know in advance
the URI of the resource to be created

rest

DELETE

erases (removes) a resource identified by a URI

it is idempotent

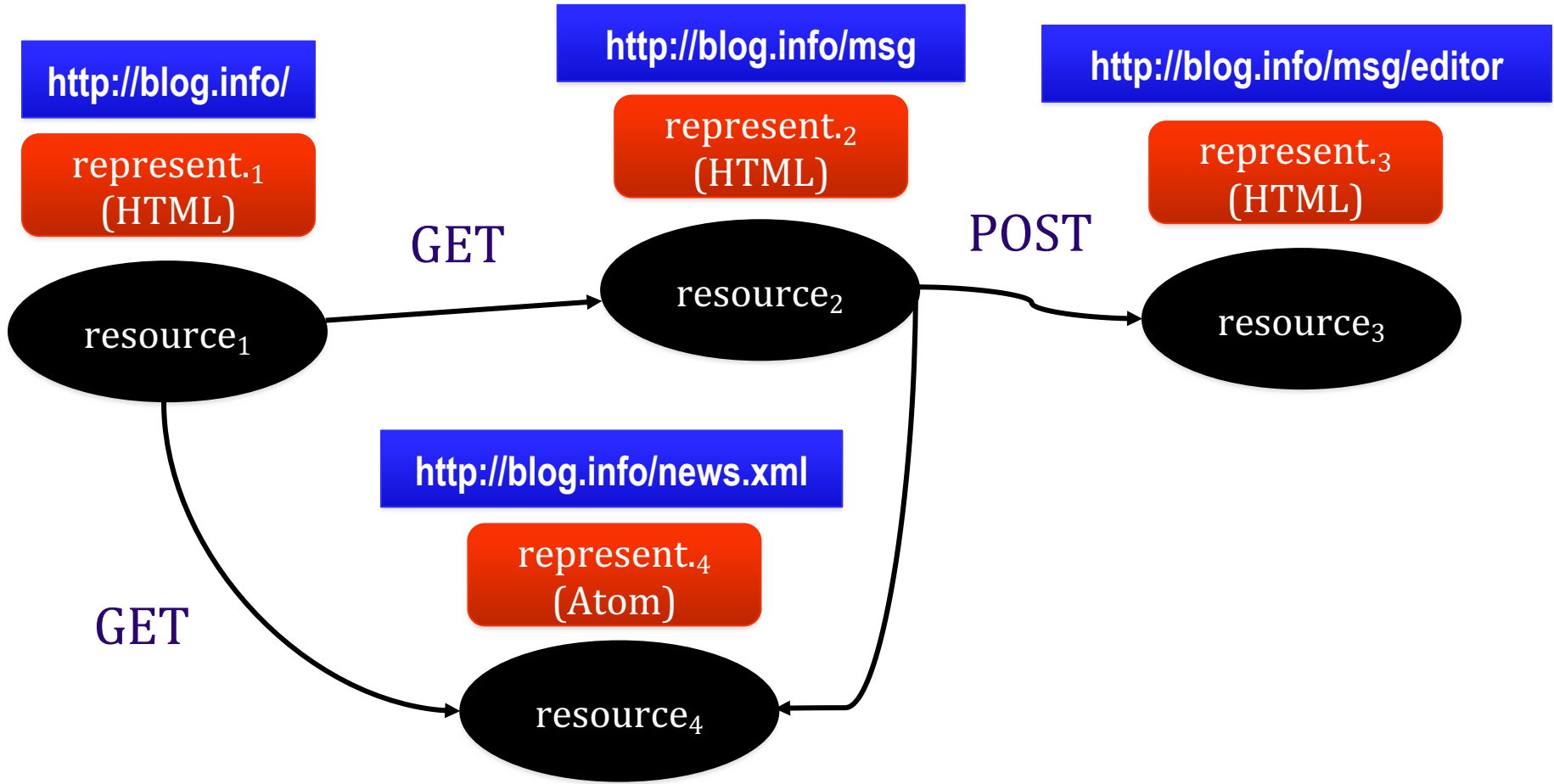
rest

OPTIONS

allows the client to determine various requirements regarding a resource (e.g., if a resource could be deleted) or the features exposed by a server (for example, the support provided by a proxy)

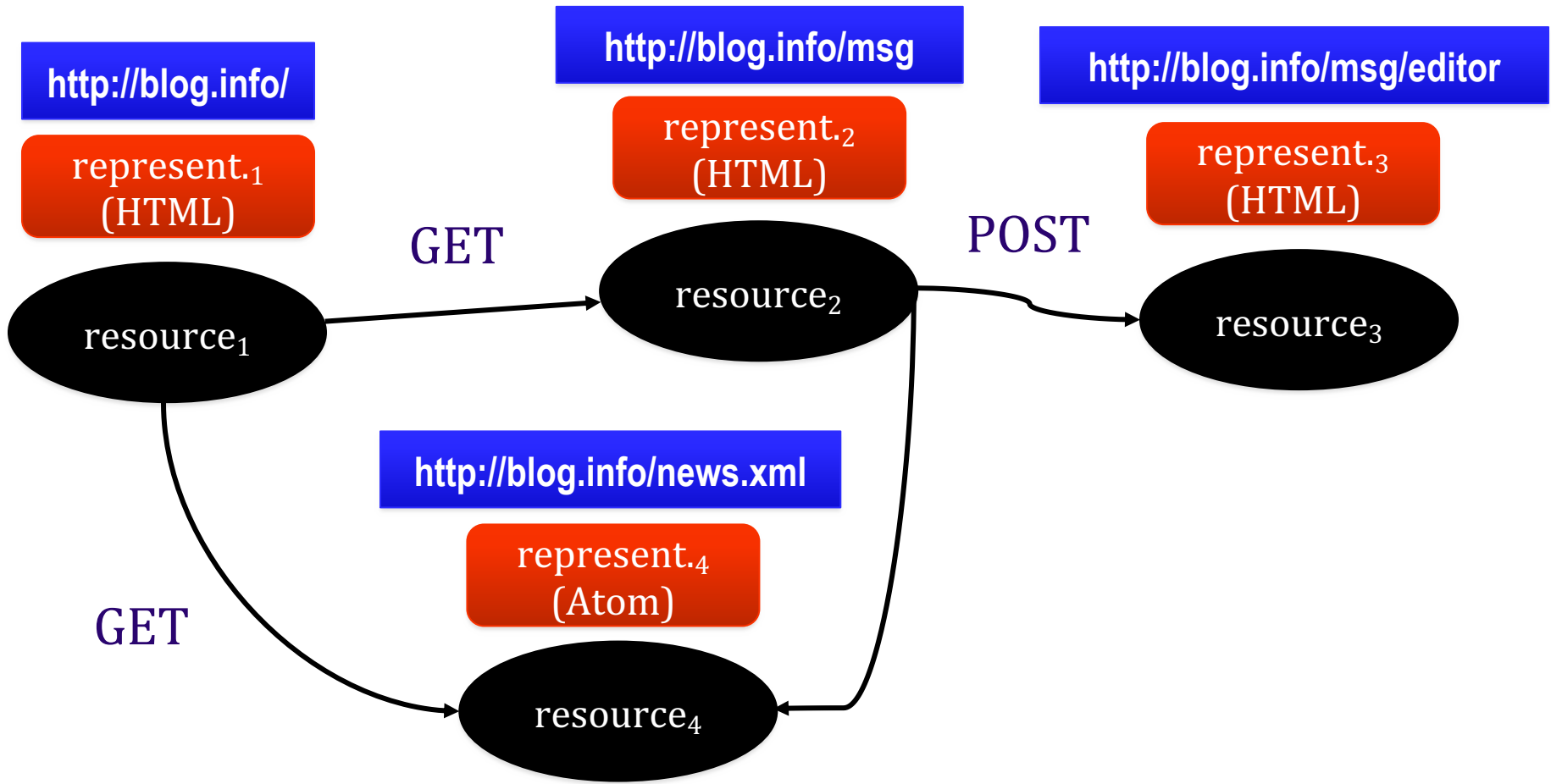
rest

Any access to a representation places the application – or the Web client – into a **state** that will be changed after a data **transfer** (accessing another representation)



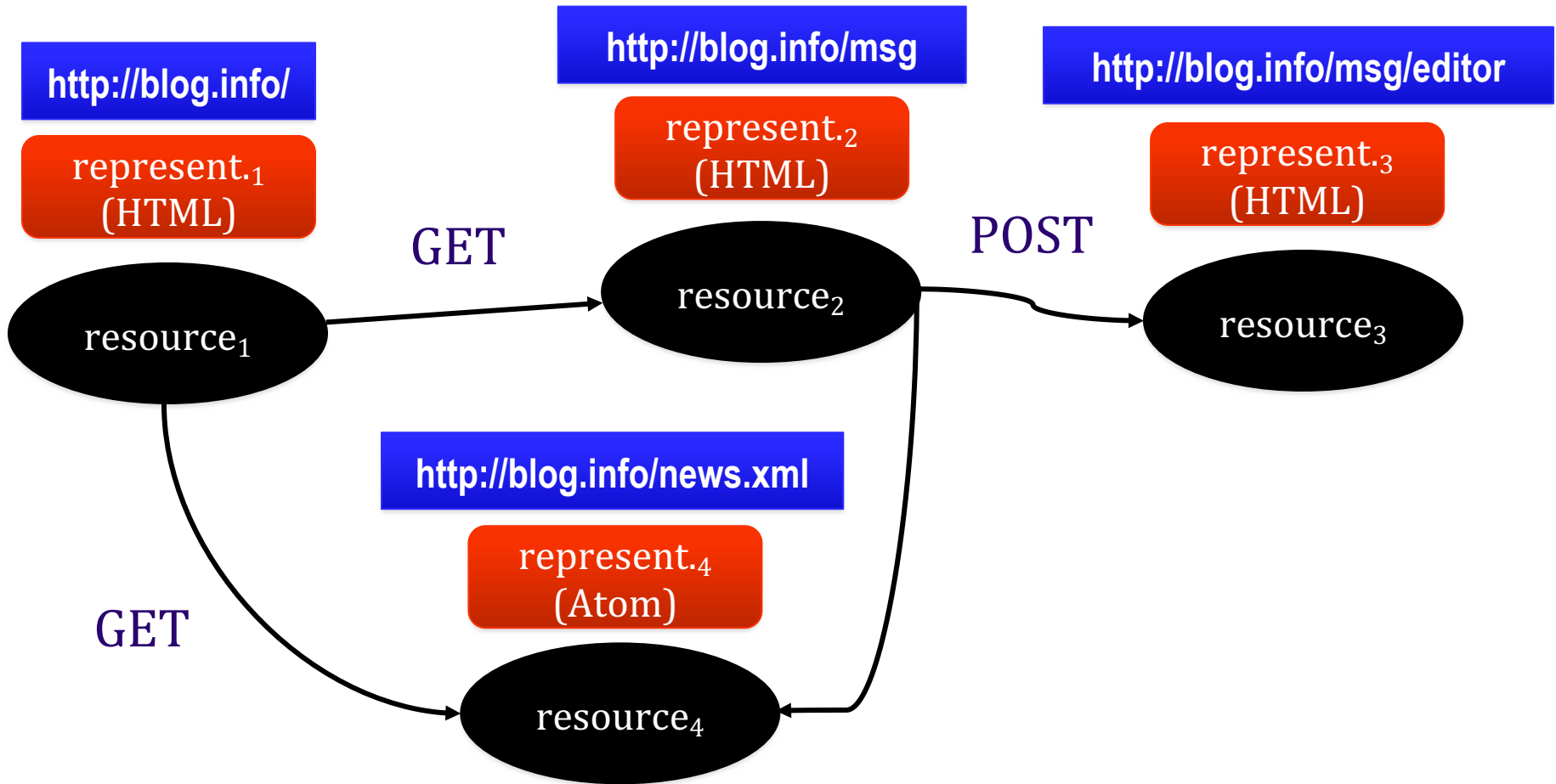
Web application $\equiv$ nondeterministic finite automaton

the transitions between states are dictated by data transfers based on the protocol's methods – usually, HTTP



based on hypermedia – the graph of connections between resources –, from a certain state (representation) transitions can be made to other states

the user/program accesses another resource via an URI



HATEOAS (Hypermedia As The Engine Of Application State)

nordicapis.com/designing-a-true-rest-state-machine/
nordicapis.com/rest-state-machine-revisited

rest

The transfer is performed through **HTTP** protocol

The representation is modeled according to a format – e.g., **JSON** or **XML** – and indicated by **media (MIME) types**

The addressability is resolved via **URI**

rest

The applications invoking functionalities (services)
consume resource representations – in pull style

rest

Each request is considered to be independent,
without considering the context

stateless server

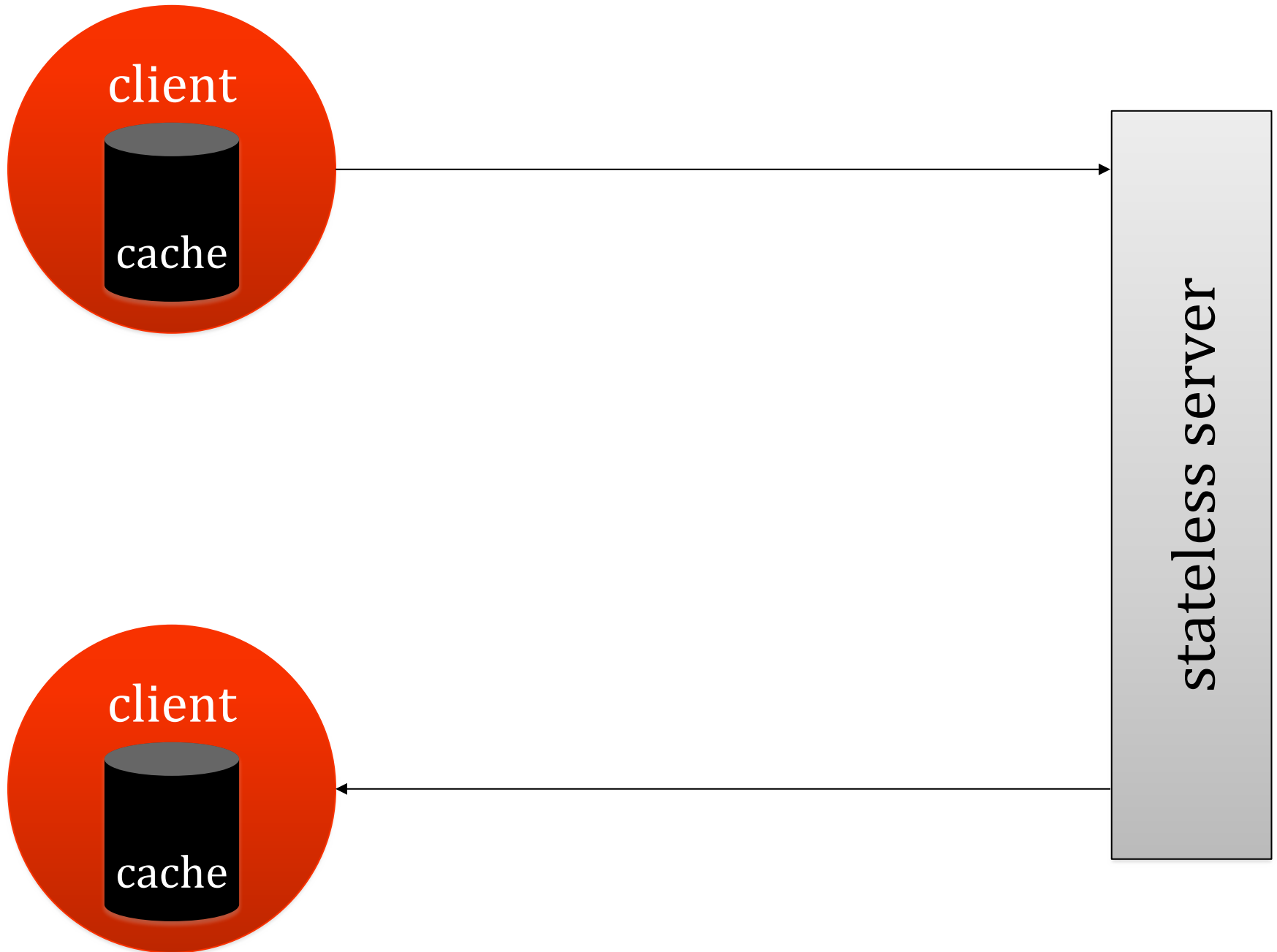
rest

Resource representations
can be temporarily stored

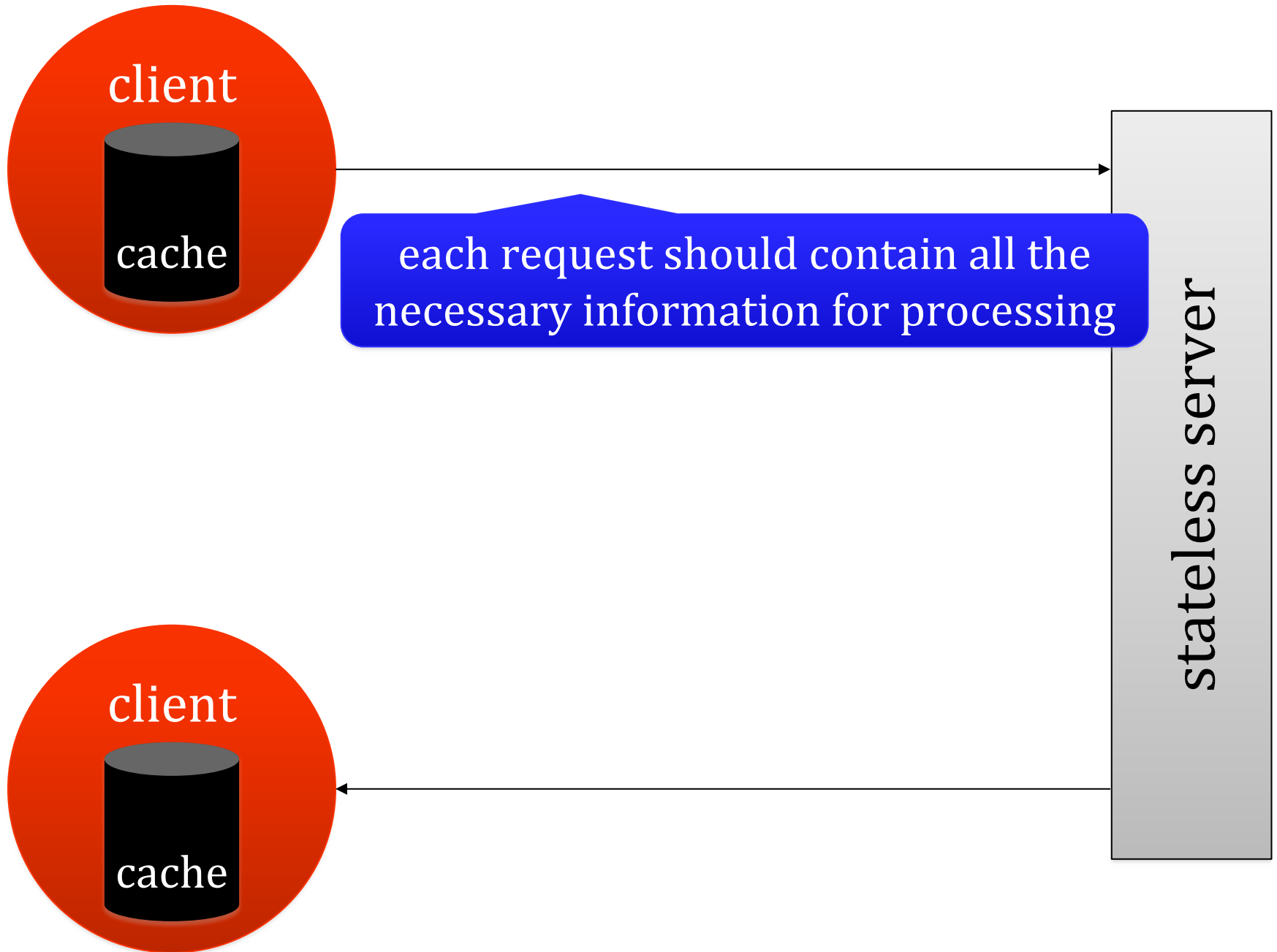
caching

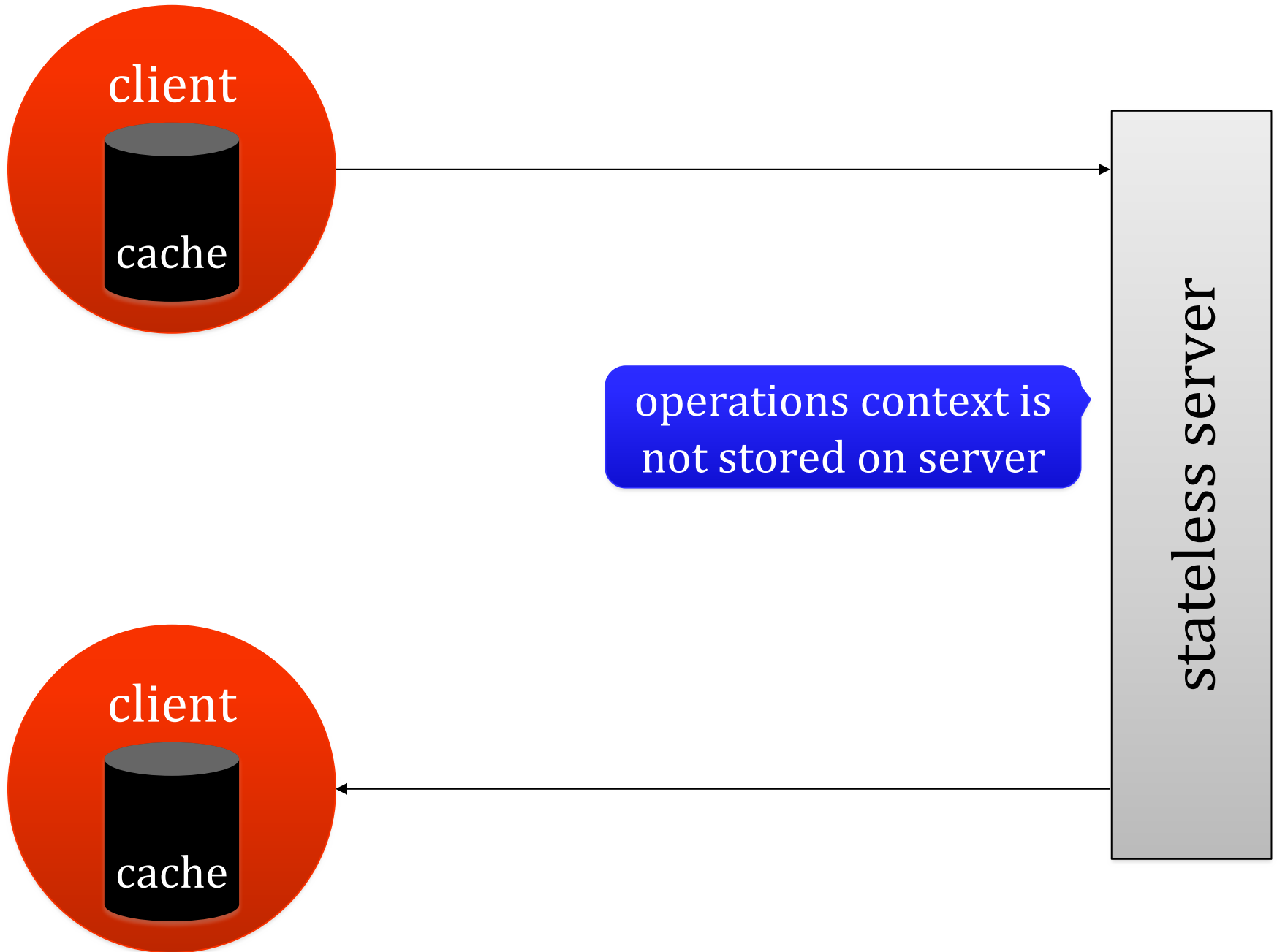
details of interest:

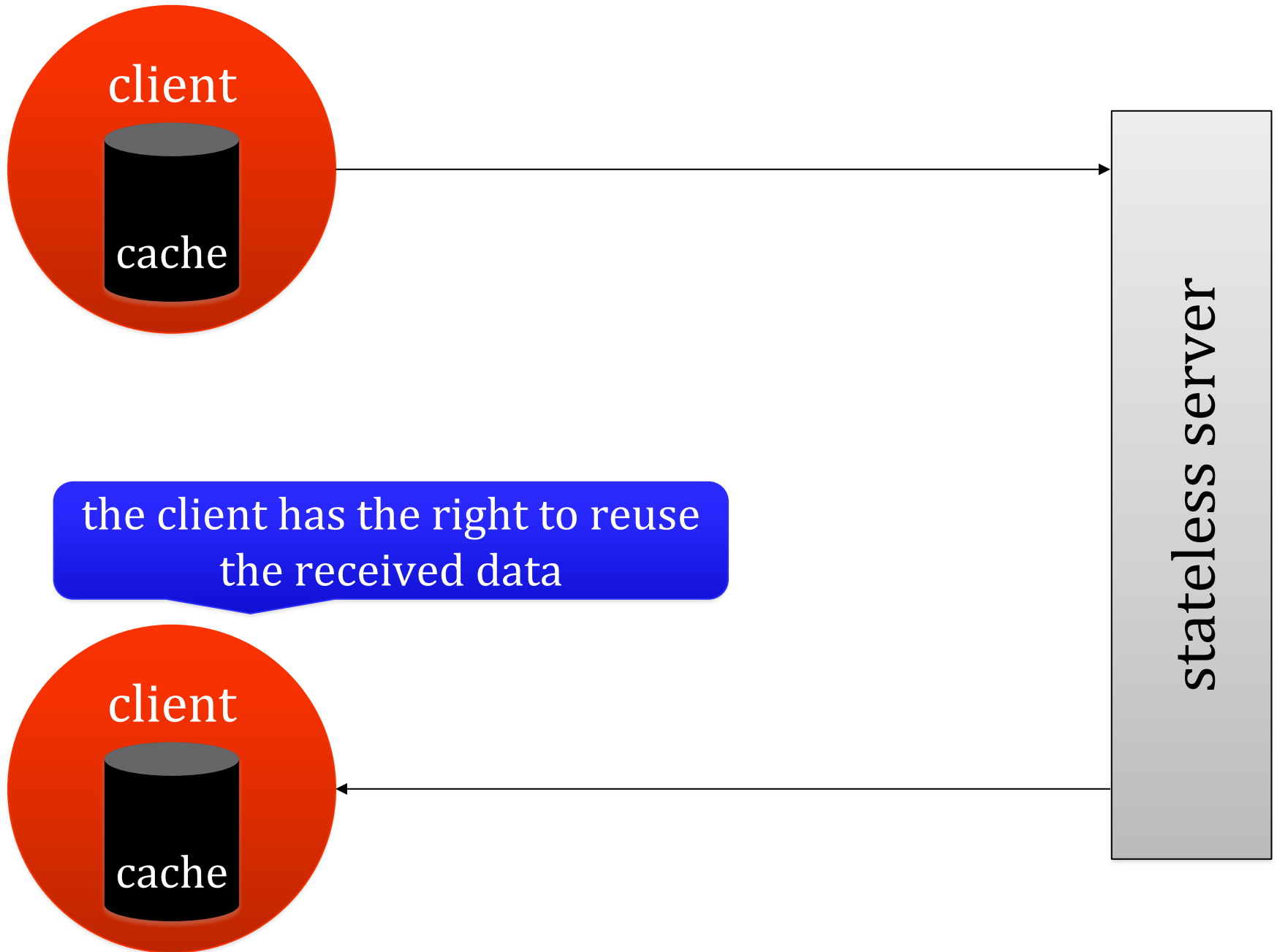
developer.mozilla.org/docs/Web/HTTP/Caching



adaptation after B. Mulloy (2012)



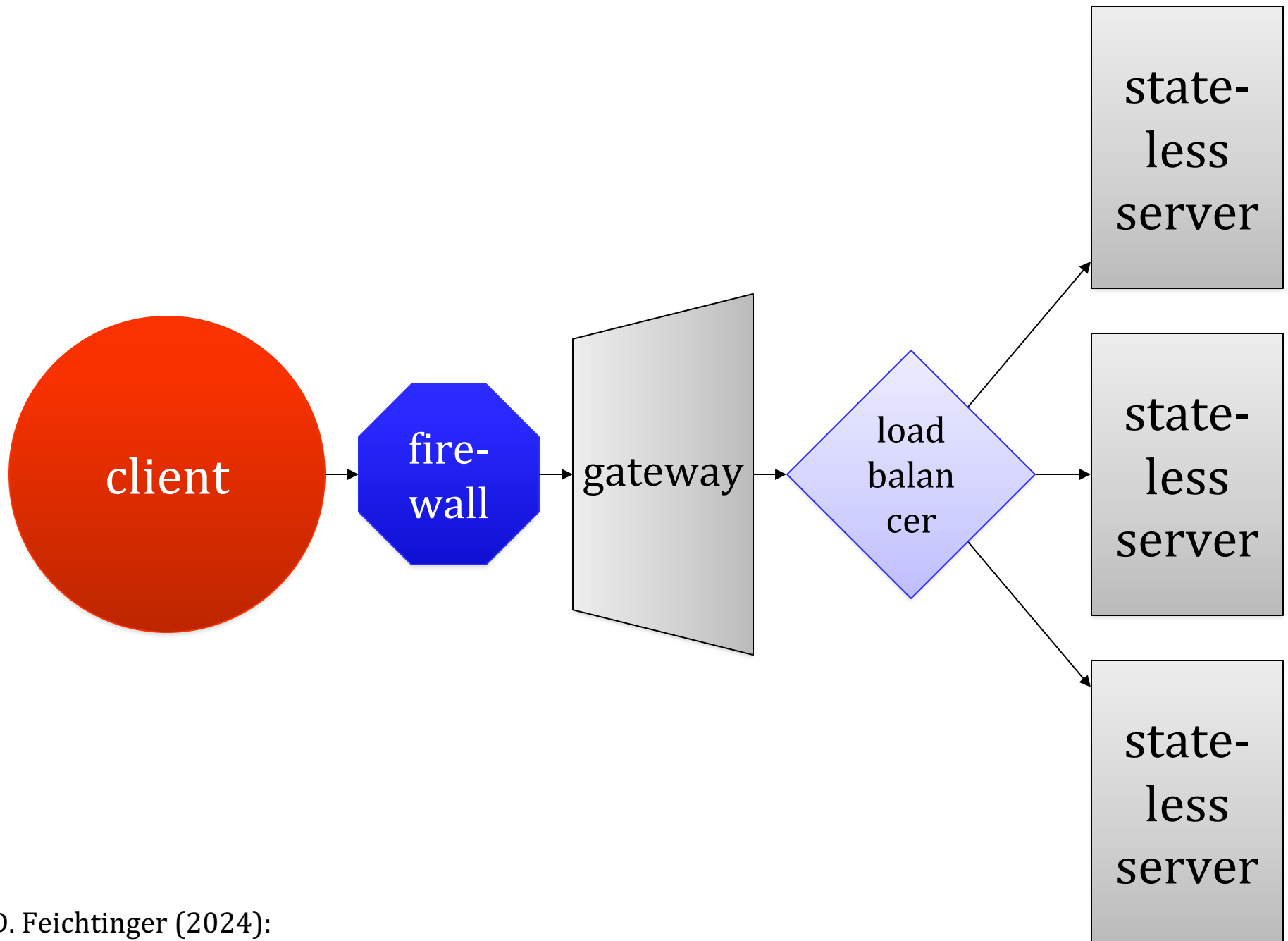


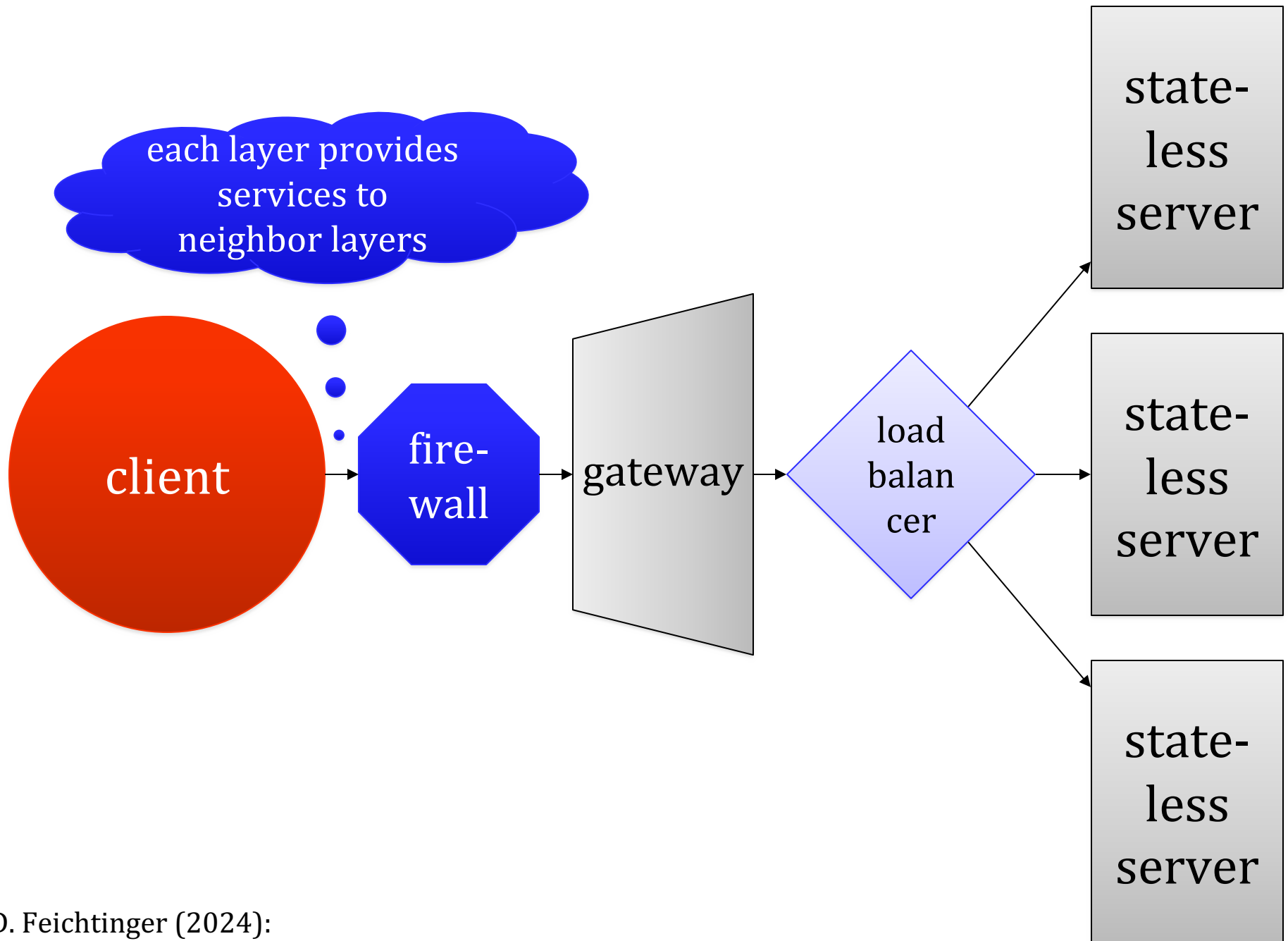


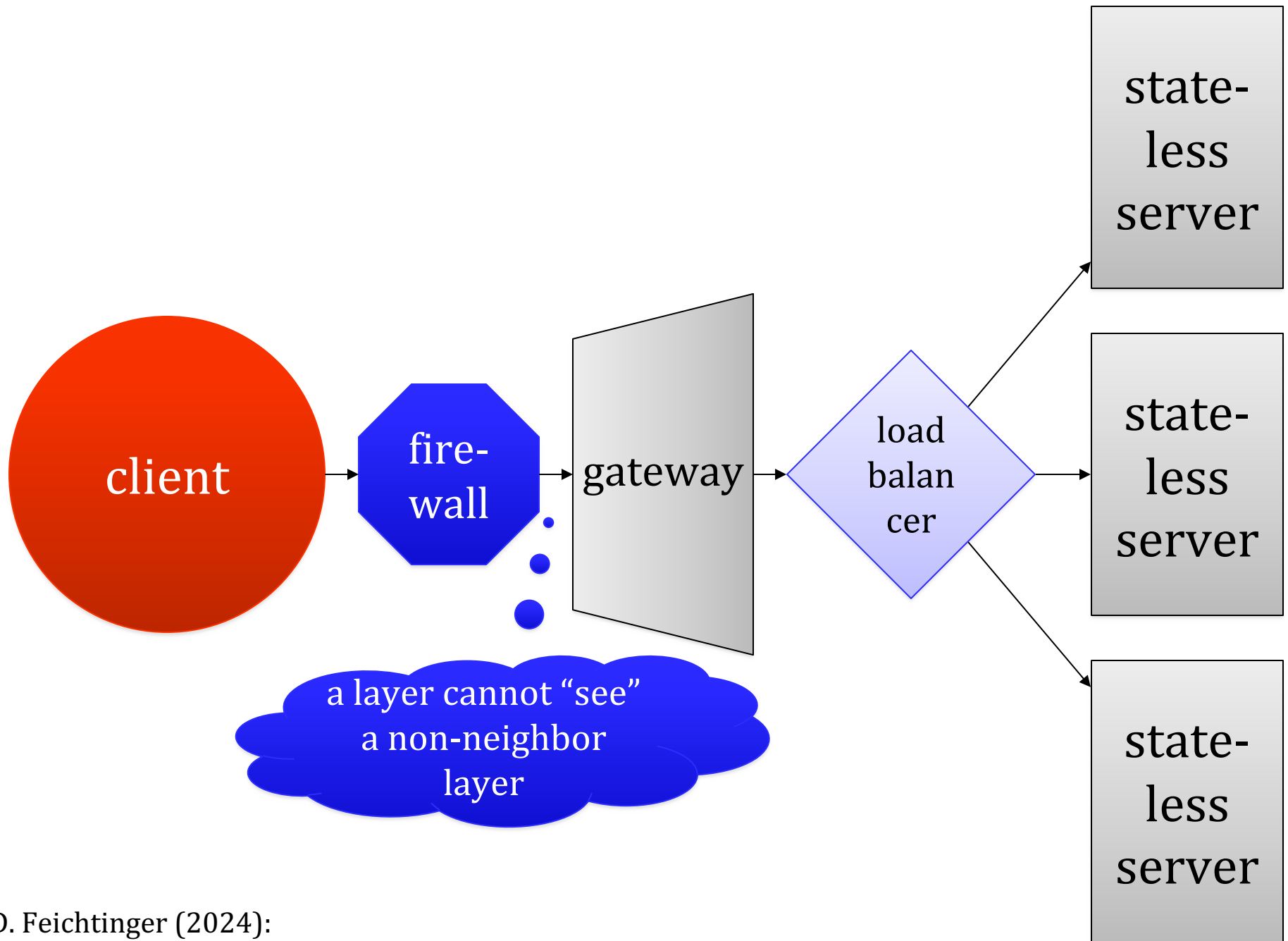
rest

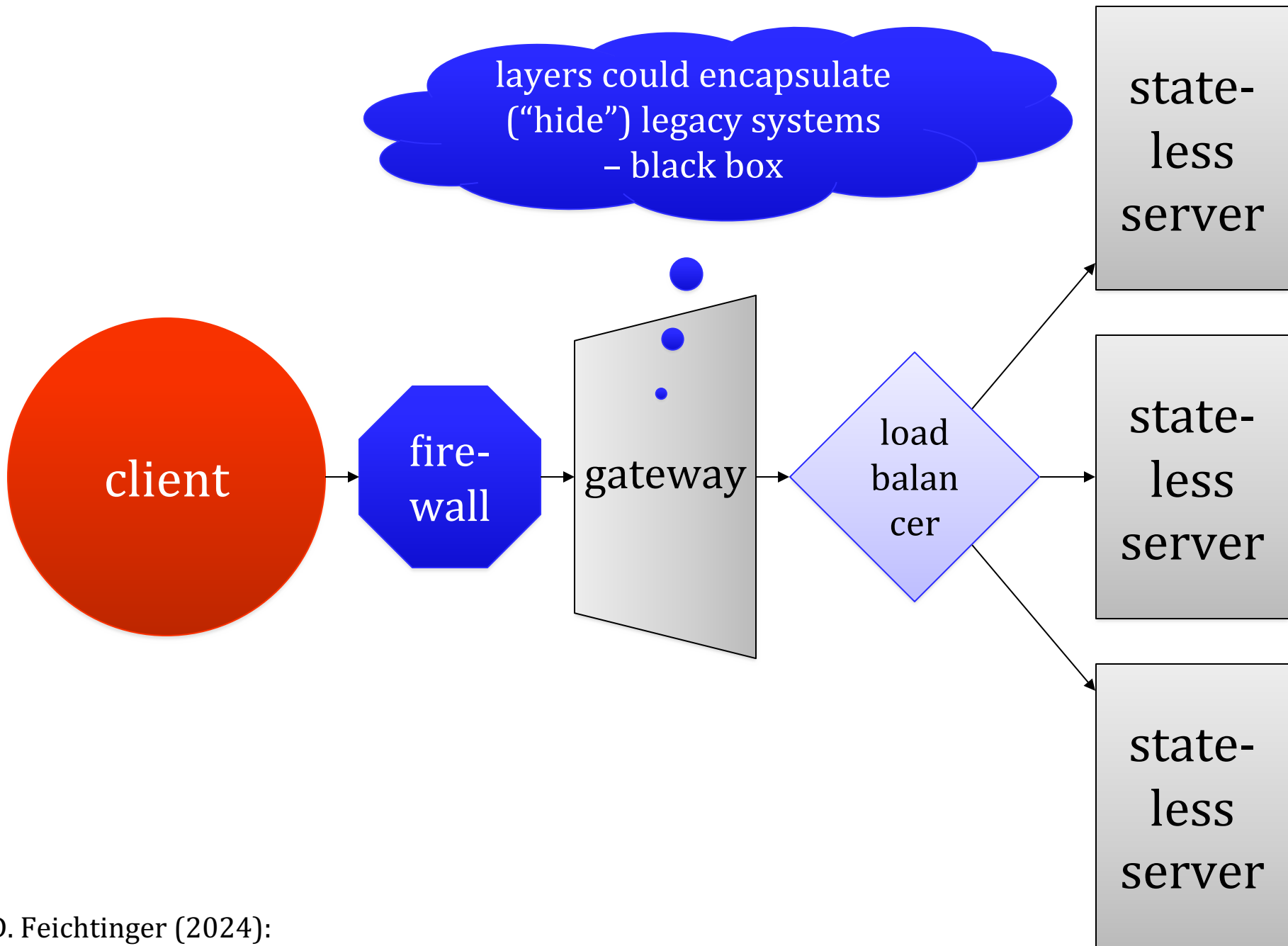
The developed Web application will adopt
a layered approach

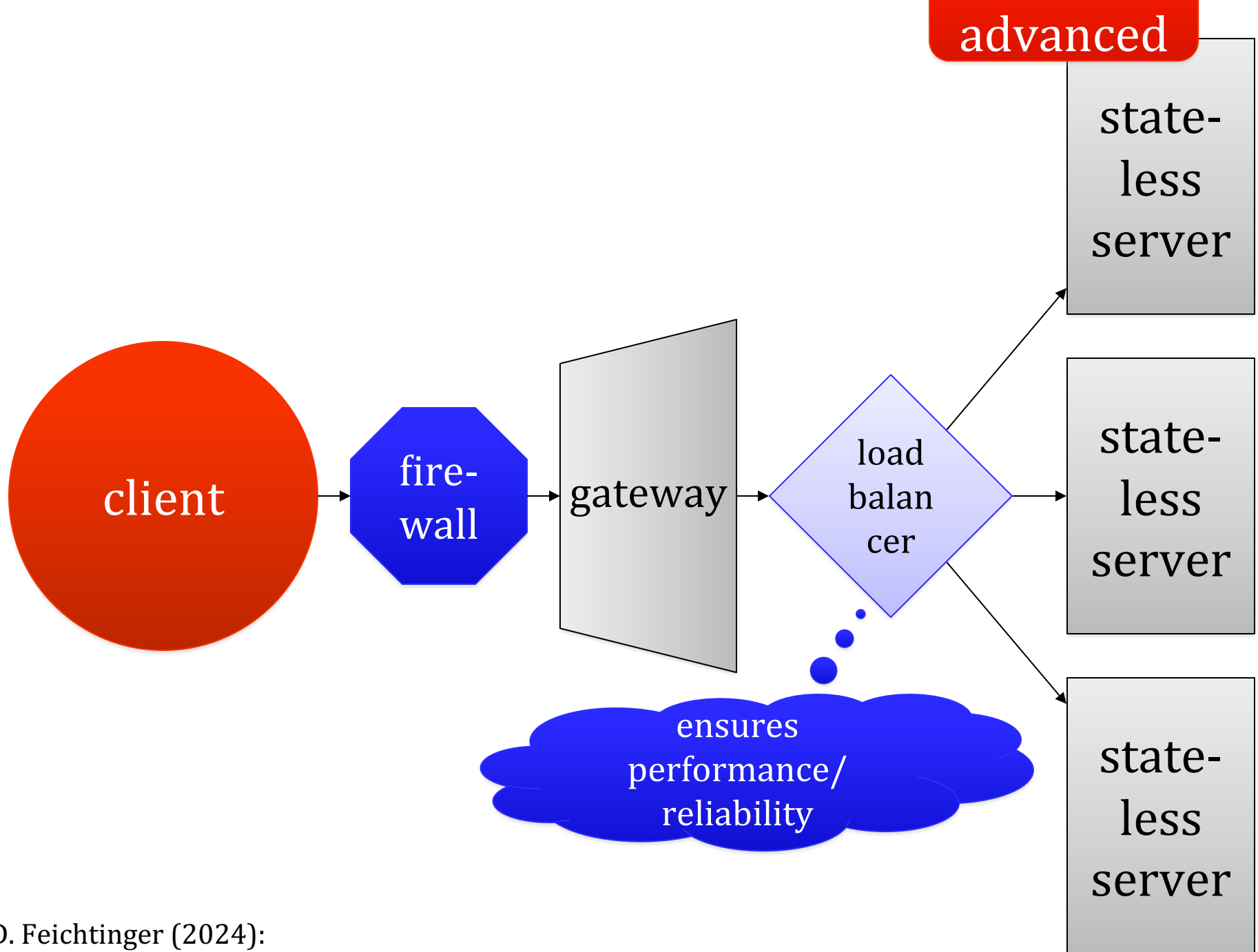
layered system











rest: example

A service to manage favorite Web addresses (bookmarks),
offering support for tags and comments

social bookmarking

alternatives: Pinterest, Raindrop.io, Reddit, etc.

rest: example

A service to manage favorite Web addresses (bookmarks),
offering support for tags and comments

functionalities regarding bookmark management:
listing (eventually, filtered by various criteria),
adding, editing, deleting, sharing

rest: example

Resource	URL	Method	Representation
Bookmark	/bookmarks/{hash}	GET	application/bookmark+xml
Bookmark	/bookmarks/{hash}	PUT	application/bookmark+xml
Bookmark	/bookmarks/{hash}	DELETE	
List of bookmarks	/bookmarks	GET	application/atom+xml
List of users	/users	GET	application/atom+xml
List of tags	/tags	GET	application/atom+xml
Main page	/	GET	application/xml

GET /bookmarks

200 OK

Content-type: application/atom+xml ..

XML response (in Atom format)
provided by the service

<?xml version="1.0"?>

<feed xmlns="http://www.w3.org/2005/Atom">

<title>Bookmarks</title>

<entry>

<title>An interesting resource</title>

digest – hash
(SHA-1, SHA-3,...)

<link

href="/bookmarks/a211528f...bdcf"/>

<summary>

http://somewhere.info/a-resource-of-interest

</summary>

</entry>

<!-- eventually, other <entry> elements... -->

</feed>

getting
bookmarks

GET /bookmarks/a211528f...bdcf

200 OK

Content-type: application/bookmark+xml

```
<bookmark>
  <title>An interesting resource</title>
  <url>http://somewhere.info/a-resource-of-interest</url>
  <user href="/users/tux">tux</user>
  <tags>
    <tag href="/tags/interesting">interesting</tag>
    <tag href="/tags/penguin">penguin</tag>
  </tags>
</bookmark>
```

accessing a bookmark:
XML response provided by the Web service

creating
a bookmark

POST /bookmarks

Content-type: application/bookmark+xml

...

201 Created

Location: /bookmarks/a211528f...bdcf

PUT /bookmarks/a211528f...bdcf

Content-type: application/bookmark+xml

...

200 OK

replacing
a bookmark

rest

Resources are named by using URIs (URLs)

Representations are interconnected through URLs

“Hypermedia as the engine of application state is a REST constraint.”

(Roy Fielding, 2014)

Intermediaries (e.g., proxy, cache, gateway) can exist between clients and resources ► performance, security,...

Data transfer could be asynchronous – Ajax et al.

previous lecture

A resource can have representations associated, that can be accessed/alterd via HTTP operations

CRUD – Create, Retrieve, Update, Delete

Operation	SQL	HTTP
Create	INSERT	PUT POST
Read (Retrieve)	SELECT	GET
Update (Modify)	UPDATE	PUT POST PATCH
Delete (Destroy)	DELETE	DELETE

practical case: **LoopBack** framework – loopback.io/doc/ operations↔REST↔implementation model↔SQL

Model create, retrieve, update, and delete operations

When you connect a model to a persistent data source such as a database, it becomes a *connected model* with a full set of create, read, update, and delete operations from the [PersistedModel](#)  class:

Operation	REST	LoopBack model method (Node API)*	Corresponding SQL Operation
Create	PUT /modelName POST /modelName	create()  *	INSERT
Read (Retrieve)	GET /modelName?filter=...	find()  *	SELECT
Update (Modify)	PUT /modelName	updateAll()  *	UPDATE
Delete (Destroy)	DELETE /modelName/ /modelID	destroyById()  *	DELETE

(*) Methods listed are just prominent examples; other methods may provide similar functionality; for example: `findById()`, `findOne()`, and `findOrCreate()`.

rest

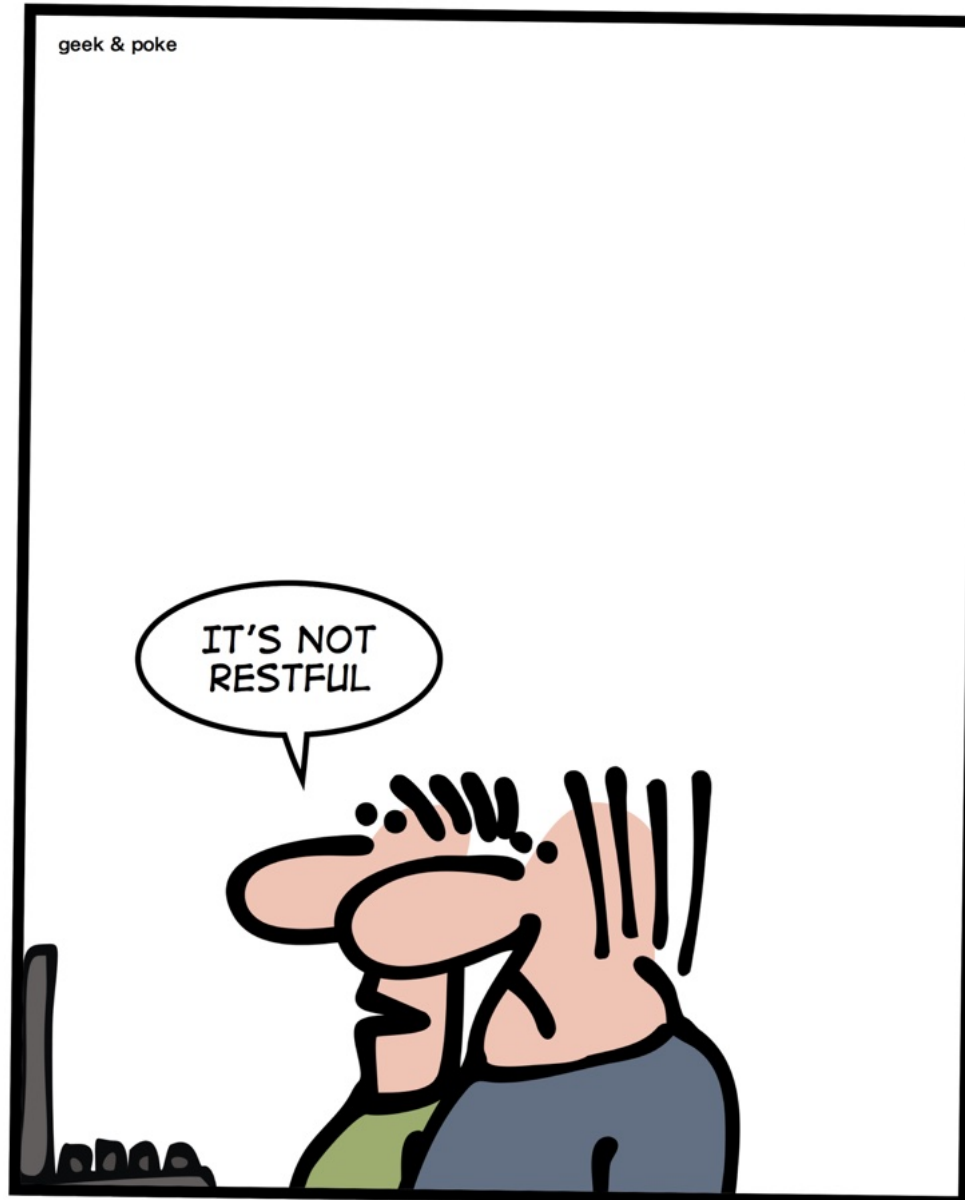
The interaction with a Web service developed according to REST can be done via an API (Application Programming Interface)



see next lecture

(instead of) break

HOW TO INSULT A DEVELOPER





Can we adopt a methodology
regarding the development of Web services
(APIs) aligned to the REST paradigm?

rest: methodology

Dividing the problem's data into
resource categories

typical classes of resources:

 **Users**

 **Documents** – alternatives: **Pictures**, **Products**, **Software**,...

 **Metadata** – e.g., **Comments**, **Formats**, **Locations**, **Platforms**, etc.

rest: methodology

Dividing the problem's data into
resource categories

real cases:

Discogs – www.discogs.com/developers/
Artists Releases Labels Versions Orders Stats

StackExchange – api.stackexchange.com
Users Answers Badges Comments Questions Revisions Tags

Unsplash – unsplash.com/documentation
Users Photos Collections Topics Stats

rest: methodology

“Naming” each resource by using URI

examples:

web.info/Users/tux

web.info/Documents/penguins-with-blue-apples

rest: methodology

“Naming” each resource by using URI

real cases:

accessing the news about a topic of interest

www.reddit.com/r/programming/

access to the Slideshare presentations of the **busaco** user

www.slideshare.net/busaco/presentations

obtaining the followers list for an authenticated user

x.com/followers

rest: methodology

Organizing resources

these resources (object instances) can be organised
into collections or stores

consult D. Denicola, Creating Truly RESTful APIs (2013)
www.slideshare.net/slideshow/creating-truly-res-tful-apis/17784394

rest: methodology

Organizing resources

collection

a catalogue of resources managed by the server
the clients can propose the alteration of a collection
the server decides the result of an operation

examples (GitHub): [/orgs/openstack/repos](#), [/orgs/openstack/events](#)

rest: methodology

Organizing resources

store

a “reserve” of resources managed by the client
(including filtering, sorting, pagination,
accessing meta-data,...)

example (GitHub): /users/openstack/repos?page=2&per_page=3

rest: methodology

Designing the **accepted** representation(s) that can be sent from the client application and the representation(s) **returned** to the client

consider common standard formats:

CSV – Comma Separated Values

JSON(-LD) – JavaScript Object Notation (-Linked Data)

XML – Extensible Markup Language

rest: methodology

Designing the **accepted** representation(s) that can be sent from the client application and the representation(s) **returned** to the client

example: JSON data requested from the service via HTTP(S)
client request – **Accept:** **application/json**
server response – **Content-Type:** **application/json**

rest: methodology

Resource integration
via hypermedia links + forms

example (GitHub):

“All resources may have one or more `*_url` properties linking to other resources. These are meant to provide explicit URLs so that proper API clients don’t need to construct URLs on their own.”

docs.github.com/en/rest#hypermedia

rest: methodology

Creating case studies

specifying the error and/or exception conditions

eventually, aspects regarding the API versions control



Users

[Show/Hide](#) | [List Operations](#) | [Expand Operations](#) | [Raw](#)

POST /Users/login Login a user with username/email and password

POST /Users/logout Logout a user with access token

GET /Users/confirm Confirm a user registration with email verification token

POST /Users/reset Reset password for a user with email

GET /Users/{id}/accessTokens/{fk} Find a related item by id for accessTokens

DELETE /Users/{id}/accessTok

PUT /Users/{id}/accessTok

GET /Users/{id}/accessTok

POST /Users/{id}/accessTokens Creates a new instance in accessTokens of this model.

DELETE /Users/{id}/accessTokens Deletes all accessTokens of this model.

GET /Users/{id}/accessTokens/count Counts accessTokens of User.

POST /Users Create a new instance of the model and persist it into the data source

PUT /Users Update an existing model instance or insert a new one into the data source

GET /Users Find all instances of the model matched by filter from the data source

GET /Users/{id}/exists Check whether a model instance exists in the data source

HEAD /Users/{id} Check whether a model instance exists in the data source

GET /Users/{id} Find a model instance by id from the data source

DELETE /Users/{id} Delete a model instance by id from the data source

PUT /Users/{id} Update attributes for a model instance and persist it into the data source

StrongLoop API

operations with specific resources – here **Users**

POST /Users

Create a new instance of the model and persist it into the data source

PUT /Users

Update an existing model instance or insert a new one into the data source

Response Class

Model | Model Schema

```
{
  "realm": "",
  "username": "",
  "credentials": "object",
  "challenges": "object",
  "email": "",
  "emailVerified": false,
  "verificationToken": "",
  "status": "",
  "created": "",
  "lastUpdated": "".
```

StrongLoop API

API interactive testing

loopback.io/api-explorer/

Response Content Type

Parameters

Parameter	Value	Description	Parameter Type	Data Type
data	Parameter content type: application/json application/json application/x-www-form-urlencoded	Model instance data	body	Model Model Schema User { realm (string, optional), username (string, optional), credentials (object, optional), challenges (object, optional), email (string), emailVerified (boolean, optional), verificationToken (string, optional), status (string, optional), created (string, optional), lastUpdated (string, optional), id (number, optional) }

Try it out!



Practical aspects of interest for developers?

rest: pragmatic view

URLs denoting resources (concepts) of interest
must be simple and intuitive

using nouns for each “thing” (entity)

rest: pragmatic view

URLs denoting resources (concepts) of interest
must be simple and intuitive

resource collections (commonly, in plural form)
/students

rest: pragmatic view

URLs denoting resources (concepts) of interest
must be simple and intuitive

resource collections (commonly, in plural form)
/students

unique identifiers for the members of a collection
/students/tuxy (concrete) vs. **/students/69** (abstract)

rest: pragmatic view

URLs denoting resources (concepts) of interest must be simple and intuitive

URLs' hierarchical structure represents the hierarchy of the resources from the modeled application domain

example (Discogs):

`/users/{name}/collection/folders/{f_id}/releases/{r_id}/instances/{i_id}`

rest: pragmatic view

The use of HTTP verbs (methods) to perform operations on (collections of) resources

resource (URI)	POST (create)	GET (access)	PUT (update)	DELETE (erase)
/students	creates a new student	lists all existing students	updates a set of students	deletes all students
/students/69 (an existing URL)	error 🙄	gets data about this student	if exists, updates; otherwise, error	deletes this student

rest: pragmatic view

Error reporting

the use of HTTP status codes

developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

typical examples:

200 OK, 204 No Content, 206 Partial Content

303 See Other, 304 Not Modified

400 Bad Request, 401 Unauthorized, 403 Forbidden,

404 Not Found, 405 Method Not Allowed

500 Internal Server Error, 503 Service Unavailable

rest: pragmatic view

Error reporting

returned messages should include useful information

example:

GitHub – provided HTTP status code: 404

```
{  
  "message": "Not Found",  
  "documentation_url": " https://docs.github.com/en/rest "  
}
```

any returned representation includes at least one URL

rest: pragmatic view

Version control of the developed API

“Never release an API without a version and make the version mandatory.” (Mulloy, 2012)

information of interest in the article J. Curry,
Introduction to API Versioning Best Practices (2017):
nordicapis.com/introduction-to-api-versioning-best-practices/

rest: pragmatic view

Version control of the developed API

specifying the version

in the HTTP header

Accept: application/vnd.heroku+json; version=3

Accept: application/vnd.discogs.v2.html+json

rest: pragmatic view

Version control of the developed API

specifying the version

in the HTTP header

in some cases, using a custom header: X-API-Version

real example: X-GitHub-API-Version:2022-11-28

incorrect versions lead to the return of the code HTTP 400

rest: pragmatic view

Version control of the developed API

specifying the version

inside the URL – eventually, as a parameter

api.cloudflare.com/client/v4/certificates/

[wss://api.openai.com/v1/realtime?model=gpt-4o-...](https://api.openai.com/v1/realtime?model=gpt-4o-...)

api.cognitive.microsofttranslator.com/translate?api-version=3.0

ec2.amazonaws.com/?Action=RunInstances&Version=2016-11-15

rest: pragmatic view

Version control of the developed API

specifying the version

continuous versioning

access using the same URI, regardless of the current version ► preferred practice (“Cool URIs don’t change”)

nordicapis.com/continuous-versioning-strategy-for-internal-apis/

rest: pragmatic view

Version control of the developed API

specifying the version

API Brownout

deactivating (a version of) an API through (intermittent)
error signaling for short periods of time

apichangelog.substack.com/p/deactivating-an-api-one-step-at-a-time
dev.to/martyndavies/what-does-api-brownout-actually-mean-2k4j

real case: Github

github.blog/changelog/2021-08-10-brownout-notice-api-authentication-via-query-parameters-for-48-hours/

rest: pragmatic view

Content pagination and providing partial responses

usual paging practice: **page** and **per_page**
/students/projects?page=2&per_page=5

alternatively, parameters such as **limit** and **offset** can be used
/students/projects?limit=33&offset=54

rest: pragmatic view

Content pagination and providing
partial responses

optional filters can be delimited by comma
`/students?fields=name,age,year,email`

rest: pragmatic view

Heterogeneity of returned representation formats
indicating the format in the URL via an optional parameter

?fmt=json (MusicBrainz)

rest: pragmatic view

Heterogeneity of returned representation formats

specifying the accepted format
in the header of HTTP request

Accept: application/json (BT Api Store)

Accept: audio/wav (IBM Watson Text to Speech)

rest: pragmatic view

Heterogeneity of returned representation formats
indicating the format in the name of the requested resource

/venue.json (Foursquare)

/forecast.xml (Weather)

advanced

Movie Reviews API

DOWNLOAD SPEC

AUTHORIZE

Try this API

Request parameters

type

picks

offset

integer

✓ by-opening-date

by-publication-date

by-title

Credentials

apikey

EXECUTE

cURL

HTTP

GET https://api.nytimes.com/svc/movies/v2/reviews/picks.json?order=by-opening-date

Accept: application/json

/movies/v2/reviews/picks.json?order=...

application/json

Raw HTTP Response

```
{
  "display_title": "Parasite",
  "mpaa_rating": "R",
  "critics_pick": 1,
  "byline": "Manohla Dargis",
  "headline": "'Parasite' Review: The Lower Depths Rise With a Vengeance",
  "summary_short": "In Bong Joon Ho's new film, a destitute family occupies",
  "publication_date": "2019-10-10",
```

the response in
JSON format

interactive queries on the movie reviews service provided
by The New York Times
developer.nytimes.com

rest: pragmatic view

The use of subdomains for different APIs
of the same service provider

example:

api.bing.microsoft.com

atlas.microsoft.com

graph.microsoft.com



How can Web resource representations
be accessed through REST?

rest: development

Current Web browsers

do not require a distinct API for HTTP access

available on any platform

REST support via **XMLHttpRequest** object,
Fetch (HTML5 API), or **WebSocket** (HTML5 API)

rest: development

Libraries/APIs implementing HTTP

allow the development of desktop, mobile,... applications

have support for server-side Web hybrid applications
(mash-ups) development

do not function in the Web browser

beware of the security problems that may arise!

rest: development – libraries

Akka HTTP (Java, Scala): doc.akka.io/libraries/akka-http

AlamoFire (Swift): github.com/Alamofire/Alamofire

Axios (Javascript, Node.js): axios-http.com

Guzzle (PHP): github.com/guzzle/guzzle

HTTP (Haskell): hackage.haskell.org/package/HTTP

http.client (Python 3): docs.python.org/3/library/http.client.html

http.rb (Ruby): github.com/httprb/http

HTTP Toolkit (multi-platform solution): httptoolkit.tech

HATEOS (PHP): williamdurand.fr/Hateoas/

Hyper (Rust): hyper.rs

rest: development – libraries

java.net.http (standard package – Java SE)

LibcURL (C; Perl, PHP, Ruby,... ports): curl.haxx.se/libcurl/

LibHTTP (C): www.libhttp.org

LWP (Perl): metacpan.org/release/libwww-perl

node-fetch (Node.js): github.com/node-fetch/node-fetch

Requests (PHP): requests.ryanmccue.info

Requests (Python): requests.readthedocs.io

restc-cpp (C++): github.com/jgaa/restc-cpp

RestSharp (.NET): restsharp.dev

Resty (Go): resty.dev

rest: development – frameworks

ASP.NET MVC + Web API (C# et al.): www.asp.net/web-api

Django, hug, restless (Python)

Express, LoopBack, Restify, Sails, Superagent (Node.js)

JAX-RS (Jakarta RESTful Web Services) spec: JSR-370
projects.eclipse.org/projects/ee4j.jaxrs

implementations: Apache CXF, Eclipse Jersey, EverRest, Jello

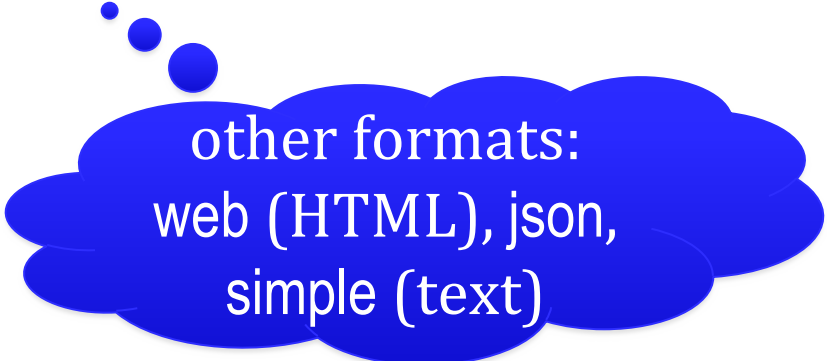
PHP micro-frameworks: Fat-Free, Lumen, Slim, Siler,...

also consult github.com/marmelab/awesome-rest

rest: implementation – example #1

Invoking a Web service for URL shortening
<https://is.gd/>

a new short URL will be created by using
<https://is.gd/create.php?format=xml&url=WebAddress>



other formats:
web (HTML), json,
simple (text)

The HTTP request that invokes the Web service:

GET /create.php?format=xml&url=profs.info.uaic.ro/sabin.buraga HTTP/1.1
Host: is.gd

The obtained response, sent by the Web server:

HTTP/1.1 200 OK

Server: nginx

Date: Mon, 11 May 2026 13:32:07 GMT+2

Content-Type: text/xml; charset=UTF-8

POX (Plain Old XML)
representation

<?xml version="1.0" encoding="UTF-8" ?>

<output>

<shorturl><https://is.gd/5sTHVh> </shorturl>

</output>

calling the Web service
via **libcurl** (PHP)

```
// initializing cURL
$c = curl_init ();
// setting the URL of the invoked Web service
curl_setopt ($c, CURLOPT_URL,
    'https://is.gd/create.php?format=xml&url=profs.info.uaic.ro/sabin.buraga');
// the result of the request will be available as a string
curl_setopt ($c, CURLOPT_RETURNTRANSFER, true);
// do not verify the digital certificate used for HTTPS data transfers
curl_setopt ($c, CURLOPT_SSL_VERIFYPEER, false);
// getting the representation provided by server (here, an XML document)
$res = curl_exec ($c); // by default, the GET method is used
// closing cURL connection
curl_close ($c);

// processing the result with DOM
$doc = new DOMDocument ();
$doc->loadXML ($res);
// getting the content of <shorturl> element
$urls = $doc->getElementsByTagName ('shorturl');
foreach ($urls as $url) {
    echo '<p>New short URL is ' . $url->nodeValue . '</p>';
}
```

rest: implementation – example 1

Invoking Web service from `https://is.gd/create.php?format=xml&url=profs.info.uaic.ro/sabin.buraga/`

Server response:

```
<?xml version="1.0" encoding="UTF-8"
standalone="yes"?><output><shorturl>https://
is.gd/5sTHVh</shorturl></output>
```

New short URL is <https://is.gd/5sTHVh>

rest: implementation – example #2

Obtaining a picture of cats corresponding to a given
HTTP status code

as in the previous example, the Web service is freely
available, without authentication, but does not follow all
REST principles (no URLs to other resources) – i.e. it is not
RESTful

<https://http.cat/>

```
define ('URL', 'https://http.cat/301');
```

```
$c = curl_init (URL); // initializing LibcURL, indicating service's URL
```

```
$opt = [ // array of options for the HTTP request
```

```
  // data will be available as strings
```

```
  CURLOPT_RETURNTRANSFER      => true,
```

```
  // do not check the digital certificate
```

```
  CURLOPT_SSL_VERIFYPEER      => false,
```

```
  // connection timeout (in seconds)
```

```
  CURLOPT_CONNECTTIMEOUT      => 10,
```

```
  // response timeout (in seconds)
```

```
  CURLOPT_TIMEOUT              => 10,
```

```
  // 4XX codes will lead to error
```

```
  CURLOPT_FAILONERROR          => true,
```

```
  // redirects are not accepted
```

```
  CURLOPT_FOLLOWLOCATION        => false ];
```

```
curl_setopt_array ($c, $opt);
```

```
$res = curl_exec ($c);
```



see the
example in the
archive

```
// HTTP status code returned by the Web service's server
$codHTTP = curl_getinfo ($c, CURLINFO_RESPONSE_CODE);
// content type offered by the server
$tip = curl_getinfo ($c, CURLINFO_CONTENT_TYPE);

// did we successfully receive an image in JPEG format?
if ($codHTTP == 200 && $tip == 'image/jpeg') {
    header ('Content-Type: ' . $tip); // send the proper MIME type
    echo $res; // display the representation of the obtained resource (here, the JPEG image)
} else {
    // send the status code returned by the service
    http_response_code ($codHTTP);
    header ('Content-Type: text/plain');
    echo 'Status code: ' . $codHTTP;

    curl_close ($c); // close the connection with the Web server
}
```



see the
example in the
archive

rest: implementation – example #2



301
Moved Permanently

All	HTML	CSS	JS	XHR	Fonts	Images	Media	WS	Other
Status	Method	File	Initiator	Type	Duration				
200	GET	rest-cat.php	document	jpeg	109 ms				
🕒	1 request	30.95 kB / 31.19 kB transferred		Finish: 109 ms					
📄	Headers	Cookies	Request	Response	Timings				
🔍 Filter Headers						Block	Resend		
▶ GET http://localhost/php/rest-cat.php									
Status		200 OK ?							
Version		HTTP/1.1							
Transferred		31.19 kB (30.95 kB size)							
Referrer Policy		strict-origin-when-cross-origin							
Request Priority		Highest							
DNS Resolution		System							
▼ Response Headers (248 B)						Raw ☒			
?	Connection: Keep-Alive								
?	Content-Type: image/jpeg								
?	Date: Mon, 05 May 2025 12:57:57 GMT								
?	Keep-Alive: timeout=5, max=100								
?	Server: Apache/2.4.63 (Unix) OpenSSL/1.1.1l PHP/8.2.28								

the rendered content + details regarding data transfer

rest: **implementation** – example #3

Accessing public data about Pokémon

resource collections:

Pokémon, Locations, Contests, Games, Moves,...

each category of resources has specific properties

e.g., any instance of Locations includes id, name, region, etc.

details pokeapi.co

Direct link to results: <https://pokeapi.co/api/v2/ability/battle-armor>

Resource for battle-armor

response available
in JSON format

```
effect_changes: [] 0 items
▼ effect_entries: [] 2 items
  ▼ 0: {} 3 keys
    effect: "Attacken können keinen kritischen Treffer landen. Diese
  ▼ language: {} 2 keys
    name: "de"
    url: "https://pokeapi.co/api/v2/language/6/"
    short_effect: "Schützt vor kritischen Treffern."
  ▼ 1: {} 3 keys
    effect: "Moves cannot score critical hits against
  ▼ language: {} 2 keys
    name: "en"
    url: "https://pokeapi.co/api/v2/language/9/"
    short_effect: "Protects against critical hits."
▶ flavor_text_entries: [] 65 items
▼ generation: {} 2 keys
  name: "generation-iii"
  url: "https://pokeapi.co/api/v2/generation/3/"
id: 4
```

```
id: 6
iso3166: "de"
iso639: "de"
name: "de"
▼ names: [] 6 items
  ▼ 0: {} 2 keys
    ▶ language: {} 2 keys
      name: "ドイツ語"
    ▶ 1: {} 2 keys
  ▼ 2: {} 2 keys
    ▼ language: {} 2 keys
      name: "fr"
      url: "https://pokeapi.co/api/v2/language/5/"
      name: "Allemand"
  ▼ 3: {} 2 keys
    ▼ language: {} 2 keys
      name: "de"
      url: "https://pokeapi.co/api/v2/language/6/"
      name: "Deutsch"
```

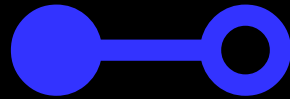
rest: development

Public services that can be consumed via REST – examples:
Amazon, Coronavirus APIs, eBay, Facebook, FitBit, GitLab,
Google, ISBNdb, LinkedIn, Microsoft, MusicBrainz,
Open Banking Tracker, OpenStreetMap, PayPal, PostNL,
Quora, Tumblr, Vimeo, Wikidata,...

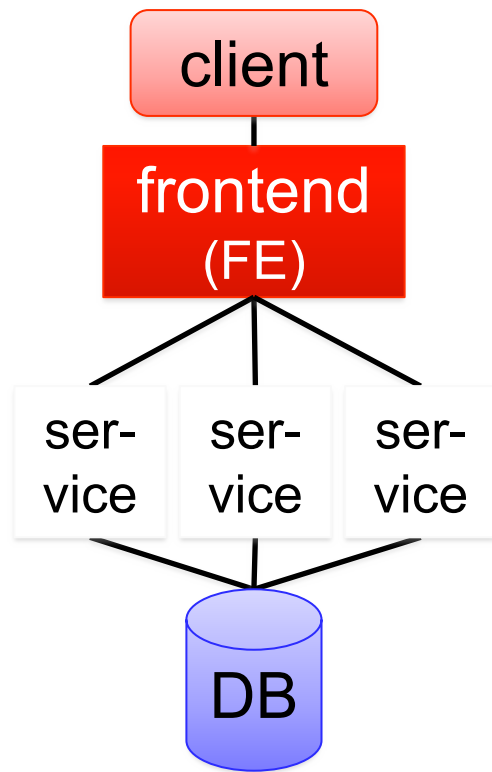
a list of public APIs:
github.com/n0shake/Public-APIs

summary

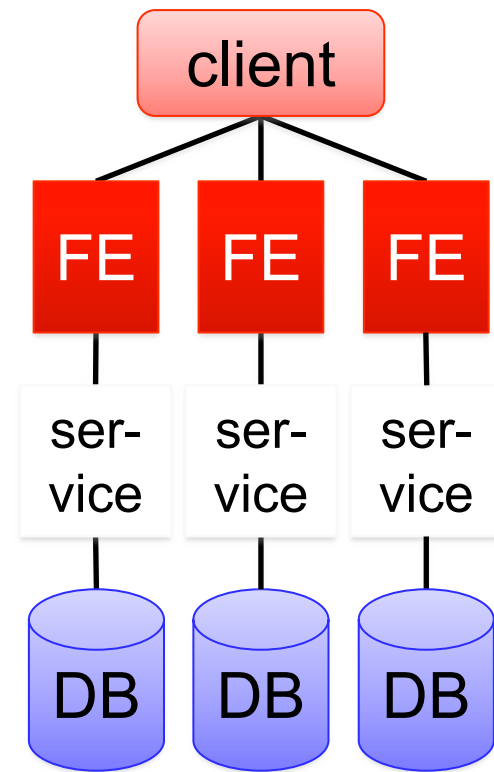
Web services development via REST



definitions, methodologies, tools, and examples



architecture based on
Web services



architecture based on
microservices

next episode:

**the development of complex Web applications:
micro-services, APIs specification, GraphQL**